CSCI-GA 2110

Programming Languages

Course Notes

Fall 2026

Instructor: Prof. Joseph Tassarotti
Recitation leaders: Seong-Heon Jung and Vishnu Satish
Notes by You Li

Acknowledgements

These notes follow the Fall 2026 offering of CSCI-GA 2110 (Programming Languages) at the Courant Institute, taught by Prof. Joseph Tassarotti, with recitations led by Seong-Heon Jung and Vishnu Satish. The course studies programming languages by *implementing* them: each new language feature is added to an interpreter written in Racket, following Krishnamurthi's *Programming Languages: Application and Interpretation* [1].

Each chapter covers one week: first the lecture, split into one section per topic, then that week's recitation. The code is the code shown in lecture and the recitation handouts; recitation solutions are the ones posted by the teaching assistants. Sections and subsections marked (SUPPLEMENT) go beyond the course material, drawing mainly on [1], [2], [3], and [5].

Notation. Code is Racket (`#lang racket`) in Chapters 1 and 2 and `#lang plai-typed` from Chapter 3 on. Interactions at the REPL are shown with a > prompt; in `plai-typed` the line starting with - is the type the checker reports, and the line after it is the value. A trace such as

```
(fact 2)
---> (* 2 (fact 1))
```

shows one evaluation step per line. In the interpreter chapters, e ranges over expressions of the language being implemented, $e[v/x]$ is e with v substituted for the free occurrences of x, and $\mathrm{FV}(e)$ is the set of free variables of e. Type names such as ArithE are set in sans serif in prose.

Contents

1 Week 1: Course Introduction and Racket Basics **5**

1.1 Studying Programming Languages . . . 6
1.1.1 Classification Is Not Understanding . . . 6
1.1.2 Studying by Implementing . . . 6
1.1.3 Course Logistics . . . 7
1.1.4 (SUPPLEMENT) Further Topics . . . 7
1.2 Expressions, Functions, and Recursion . . . 8
1.2.1 Racket Is an Expression Language . . . 8
1.2.2 Defining Functions . . . 8
1.2.3 Conditionals . . . 8
1.2.4 Recursion . . . 8
1.2.5 Tail Recursion . . . 9
1.2.6 (SUPPLEMENT) Further Topics . . . 9
1.3 Pairs and Lists . . . 11
1.3.1 Pairs . . . 11
1.3.2 Lists Are Nested Pairs . . . 11
1.3.3 Reversing a List . . . 12
1.3.4 (SUPPLEMENT) Further Topics . . . 12
1.4 Recitation 1: Functional Programming in Racket . . . 13
1.4.1 Expressions and Functions . . . 13
1.4.2 Specifications . . . 14
1.4.3 Recursion: Scrolling Through Lights . . . 14
1.4.4 Lists and Higher-Order Programming . . . 15
1.4.5 (SUPPLEMENT) Further Topics . . . 17

2 Week 2: Higher-Order Functions, Scope, and State **19**

2.1 Lists and Accumulators . . . 20
2.1.1 `snoc`: Adding to the End . . . 20
2.1.2 Reversing, Correctly and Incorrectly . . . 20
2.1.3 Aggregating a List . . . 21
2.1.4 Named `let`: Hiding the Helper . . . 22
2.1.5 (SUPPLEMENT) Further Topics . . . 22
2.2 Structs and Trees . . . 23
2.2.1 `struct` . . . 23
2.2.2 Binary Trees . . . 23
2.2.3 `match` . . . 24
2.2.4 (SUPPLEMENT) Further Topics . . . 24

2.3 Higher-Order Functions . . . 25
2.3.1 Functions Are Values . . . 25
2.3.2 `map` and `filter` . . . 25
2.3.3 Folds . . . 26
2.3.4 (SUPPLEMENT) Further Topics . . . 27
2.4 Scope and State . . . 29
2.4.1 `let` and `let*` . . . 29
2.4.2 Lexical Scope . . . 29
2.4.3 Pure Code and Side Effects . . . 30
2.4.4 Boxes: State You Can See . . . 31
2.4.5 (SUPPLEMENT) Further Topics . . . 31
2.5 Recitation 2: Boolean Expressions . . . 33
2.5.1 The Language and Its Representation . . . 33
2.5.2 An Interpreter for bexp . . . 34
2.5.3 An Optimization Pass: De Morgan . . . 34
2.5.4 Testing the Optimizer Two Ways . . . 36
2.5.5 (SUPPLEMENT) Further Topics . . . 37

3 Week 3: Types and a First Interpreter 38
3.1 Types in plai-typed . . . 39
3.1.1 Every Value Has a Type . . . 39
3.1.2 Type Inference and Annotations . . . 39
3.1.3 Function Types . . . 40
3.1.4 Lists Are Homogeneous . . . 41
3.1.5 Pairs . . . 41
3.1.6 Polymorphic Types . . . 41
3.1.7 Why Insist on Homogeneous Lists? . . . 42
3.1.8 (SUPPLEMENT) Further Topics . . . 42
3.2 `define-type` and `type-case` . . . 43
3.2.1 Defining a Type with Variants . . . 43
3.2.2 `struct` vs. `define-type` . . . 43
3.2.3 `type-case`: Taking a Value Apart . . . 44
3.2.4 Exhaustiveness . . . 44
3.2.5 (SUPPLEMENT) Further Topics . . . 45
3.3 S-Expressions and the Calculator . . . 46
3.3.1 Quote and Symbols . . . 46
3.3.2 S-Expressions . . . 46
3.3.3 Concrete and Abstract Syntax . . . 46
3.3.4 The Calculator . . . 47
3.3.5 What Malformed Input Does . . . 48
3.3.6 (SUPPLEMENT) Further Topics . . . 49
3.4 Recitation 3: Typed Boolean Expressions and Pattern Matching . . . 50
3.4.1 Part A: Boolean Expressions, Typed . . . 50
3.4.2 Part B: Pattern Matching with `match` . . . 52
3.4.3 `match` vs. `type-case` . . . 54
3.4.4 (SUPPLEMENT) Further Topics . . . 54

4 Week 4: Names and Functions **55**
4.1 Names: `let` and Substitution . . . 56
4.1.1 The Plan . . . 56
4.1.2 Two New Variants . . . 56
4.1.3 Eager Substitution . . . 57
4.1.4 Shadowing . . . 57
4.1.5 `eval-eager` . . . 58
4.1.6 (SUPPLEMENT) Further Topics . . . 58
4.2 Lazy Substitution and Variable Capture . . . 59
4.2.1 Lazy Substitution . . . 59
4.2.2 Is Lazy More Efficient? . . . 59
4.2.3 Where Eager and Lazy Differ . . . 60
4.2.4 Variable Capture . . . 60
4.2.5 (SUPPLEMENT) Further Topics . . . 61
4.3 Capture-Avoiding Substitution . . . 62
4.3.1 Rename, Then Substitute . . . 62
4.3.2 `gensym`: Fresh Names . . . 62
4.3.3 `subst-lazy-v2` . . . 63
4.3.4 (SUPPLEMENT) Further Topics . . . 63
4.4 Functions: `lambda` and Application . . . 65
4.4.1 Syntax . . . 65
4.4.2 `lambda` in Substitution . . . 66
4.4.3 A New Result Type . . . 66
4.4.4 `eval-lazy-v2` . . . 66
4.4.5 Capture, Again . . . 67
4.4.6 (SUPPLEMENT) Further Topics . . . 68
4.5 Recitation 4: Pattern Matching in the Interpreter . . . 69
4.5.1 The New Form . . . 69
4.5.2 Types . . . 69
4.5.3 Parsing . . . 70
4.5.4 Substitution . . . 71
4.5.5 Evaluation . . . 71
4.5.6 (SUPPLEMENT) Further Topics . . . 73

Chapter 1

Week 1: Course Introduction and Racket Basics

Lecture 1: September 8, 2026. Recitation 1: September 11, 2026.

Supplementary reading:

- [1] Chapter 1 (Introduction).
- [4] Chapters 2 (Racket Essentials) and 3.8–3.9 (Pairs and Lists).
- [2] Part I, especially the design recipe (Section 3).
- [3] Sections 1.1 and 1.2.1 (the substitution model; linear recursion and iteration).

The first week explains how this course approaches programming languages, by building interpreters for small languages, and teaches enough Racket to start. This chapter covers:

- Section 1.1: why study programming languages, why classifying them by paradigm is the wrong place to start, and why the course implements interpreters;
- Section 1.2: Racket as an expression language, `define`, `cond`, recursion, and tail recursion with an accumulator;
- Section 1.3: pairs, lists, and the two-case shape of list functions;
- Section 1.4: Recitation 1, practicing function composition, specifications, recursion, and list functions on a traffic-light example.

1.1 Studying Programming Languages

1.1.1 Classification Is Not Understanding

The obvious way to organize the subject is by classification. Languages are sorted by *paradigm* (functional, imperative, object-oriented), by *abstraction* (high-level or low-level), by *representation* (text or graphical), and so on. A common course design follows from it: survey the categories, pick three or four languages from different ones, learn their basics, and write programs that highlight each one's features. That is not a bad idea, and you would learn a lot, but it is not the approach of this course.

The lecture makes the point with biological taxonomy. "Ape" sounds like a natural category: large, tailless primates such as the orangutan, the gorilla, and the chimpanzee. But when species are grouped by common descent, the group that contains all three also contains humans, and chimpanzees are closer to humans than to orangutans (Figure 1.1). A classification by surface features does not match the underlying structure. Sensible classification has to come *after* understanding the principles.

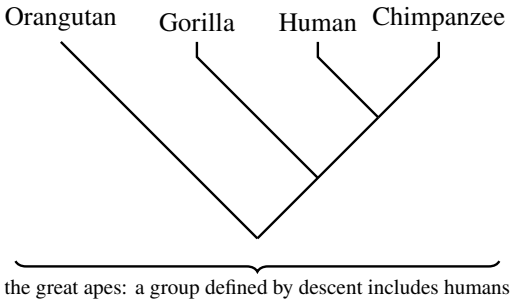


Figure 1.1: Grouping by common descent. Humans sit inside the great apes, not beside them (after slide 7).

The course's claim is that the same *building blocks* appear in many programming languages: functions, names and scope, data structures, state, types, control. It is better to understand these building blocks directly than to memorize which languages have which features.

Key Point 1 (Two false dichotomies)

- *"Functional" vs. "imperative" languages.* There is no dichotomy. A single language can have functional features, such as first-class functions, and imperative ones, such as mutable state. Racket, OCaml, Python, and JavaScript all have both.
- *"Interpreted" vs. "compiled" languages.* Interpreting and compiling describe an *implementation*, not a language. Any language can have both interpreters and compilers, so neither is a property of the language.

1.1.2 Studying by Implementing

To study the building blocks, we want to be precise and formal. One way is mathematical: give a theory of what programs mean and how they behave (formal semantics). This course takes a second way: it *implements* programming languages. To implement a feature you must decide exactly what it does in every case, so writing the implementation forces the precision that a mathematical definition would.

There are two strategies for implementing a language (Figure 1.2):

- A *compiler* translates a program in one language into another language that can be executed.

- An *interpreter* is a program that takes a program as input and directly executes it.

This course builds interpreters. Compilers are the subject of Compiler Construction (CSCI-GA 2130).

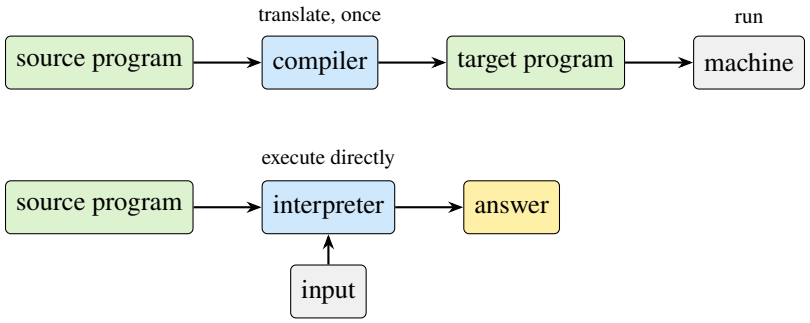


Figure 1.2: The two implementation strategies. This course writes interpreters.

1.1.3 Course Logistics

- **Staff.** Instructor: Joseph Tassarotti. Recitation leaders: Seong-Heon Jung and Vishnu Satish.
- **Textbook.** Krishnamurthi, *Programming Languages: Application and Interpretation*, 2nd edition [1], free online and linked from the syllabus.
- **Language.** The interpreters are written in Racket, for two reasons: Racket has many of the core features we want to study, so writing it is practice in them; and Racket was designed for implementing programming languages.
- **Grading.** Final exam 50%, homework 50%.
- **Homework.** Five assignments of two to three weeks each, done individually. Assignment 1 is a Racket warm-up, pure programming, auto-graded. Assignments 2–5 combine programming with conceptual questions. Up to 72 hours of lateness are allowed in total across all assignments; see the syllabus for details.

1.1.4 (SUPPLEMENT) Further Topics

Definitional interpreters. The idea of explaining a language by writing an interpreter for it is old. Reynolds [6] called such programs *definitional interpreters*: the interpreter, written in a language we already understand, *is* the definition of the new language. The method has a catch Reynolds also pointed out: if the defining language has a feature (say, eager evaluation or higher-order functions), the defined language can silently inherit it. Much of this course is about *not* inheriting features by accident. In Section 2.5, for example, our language's `and` is implemented by Racket's `and`, so it short-circuits exactly when Racket's does.

Language vs. implementation, in practice. Python is usually run by CPython, which compiles source to bytecode and then interprets the bytecode; PyPy runs the same language with a just-in-time compiler. JavaScript engines interpret first and compile hot code later. Racket itself compiles to machine code (on the Chez Scheme back end) while giving you an interactive REPL. "Compiled vs. interpreted" describes none of these languages.

1.2 Expressions, Functions, and Recursion

1.2.1 Racket Is an Expression Language

A Racket program is not a list of steps to perform; it is an expression, and running it means simplifying that expression until only a value is left. Every construct below, including conditionals and function bodies, is an expression that produces a value.

Every call is written the same way: the function first, then its arguments, all inside parentheses. So addition is `(+ 3 x)`, not `3 + x`, and calling `foo` on 4 is `(foo 4)`.

1.2.2 Defining Functions

`(define (name argument ...) body)` gives a name to a function.

```
(define (foo x)
  (+ 3 x))
```

There is no `return` keyword, because there is nothing to return *from*: the body is a single expression, and its value is the answer. `(foo 4)` evaluates to `7`.

1.2.3 Conditionals

`cond` chooses between answers. It takes clauses written `[test result]`, tries the tests top to bottom, and returns the result of the first test that is true. `else` is the catch-all at the bottom.

```
(define (my-abs x)
  (cond
    [(> x 0) x]
    [(< x 0) (- x)]
    [else 0]))
```

`cond` is an *expression*, not a statement. It produces a value, which is why it can be the entire body of `my-abs`: there is no assignment and nothing to return, and the value of the `cond` is the value of the function. With one argument, `(- x)` is negation. Only the chosen branch is evaluated, so a branch that would divide by zero is harmless as long as its test is false.

1.2.4 Recursion

Recursion is how repetition is done. A recursive function needs two things:

- a *base case* that answers directly, without recurring, and
- a *recursive case* that makes the problem smaller, in a way that eventually reaches the base case, and trusts the function itself with the rest.

```
(define (fact x)
  (cond
    [(= x 0) 1]
    [else (* x (fact (- x 1)))]))
```

The way to check a recursive function by hand is to rewrite the expression one step at a time:

```
(fact 3)
```

```
---> (* 3 (fact 2))
---> (* 3 (* 2 (fact 1)))
---> (* 3 (* 2 (* 1 (fact 0))))
---> (* 3 (* 2 (* 1 1)))
---> 6
```

The `*` sits *outside* the recursive call, so no multiplication can happen on the way down. Each level waits for the one below it, and every level stays on the stack until the recursion bottoms out. The trace grows a tower of pending multiplications.

1.2.5 Tail Recursion

The work can be rearranged so that the multiplying happens on the way *down*. Pass the answer-so-far along as an extra argument, an *accumulator*:

```
(define (fact-helper x acc)
  (cond
    [(= x 0) acc]
    [else (fact-helper (- x 1) (* acc x))]))

(define (fact-tail x)
  (fact-helper x 1))
```

The accumulator starts at 1, the value that leaves a product unchanged; a sum would start at 0.

Definition 2 (Tail call)

A call is a *tail call* if it is the last thing the calling function does: its result is returned unchanged, with no work left pending. Racket guarantees that a tail call reuses the caller's stack frame instead of pushing a new one.

The recursive call in `fact-helper` is a tail call, so the function runs in constant stack space, like a loop:

```
(fact-helper 3 1)
---> (fact-helper 2 3)
---> (fact-helper 1 6)
---> (fact-helper 0 6)
---> 6
```

Nothing is ever pending, because `acc` is a finished number at every step. Since `fact-helper` takes two arguments and `fact` one, the wrapper `fact-tail` keeps the interface the same. Hiding the helper is a recurring theme; Section 2.1 does it with a named `let`.

1.2.6 (SUPPLEMENT) Further Topics

The substitution model. The step-by-step traces above follow the *substitution model* of evaluation [3, Section 1.1.5]: to apply a function, evaluate its arguments, substitute the values for the parameters in the body, and continue with the body. This is a valid way to reason about a program exactly when the program is pure (Section 2.4). It is also, almost literally, the first interpreter we will build in Chapter 4, where `let` and function application are implemented by a function named `subst`.

Recursive process vs. iterative process. SICP distinguishes the *procedure*, which may be recursive syntactically, from the *process* it generates. `fact` generates a linear recursive process (stack depth n);

`fact-helper` is also recursive as a procedure but generates an iterative process (constant depth). In a language with guaranteed tail calls, there is no need for special loop syntax: a loop is a tail-recursive function.

Tail calls in other languages. Scheme and Racket require proper tail calls, so the guarantee holds for *every* tail call, not only self-calls. Many languages do not: CPython deliberately does not eliminate tail calls (the stack depth is capped at about 1000 by default), and Java and Rust do not guarantee it either. A C compiler may optimize tail calls at higher optimization levels, but the language does not promise it.

1.3 Pairs and Lists

1.3.1 Pairs

`cons` glues any two values into a *pair*, and `car` and `cdr` take the pair apart:

```
> (cons 3 4)
'(3 . 4)
> (car (cons 1 2))
1
> (cdr (cons 1 2))
2
```

The names `car` and `cdr` are historical, from the registers of the IBM 704 on which Lisp was first implemented. `first` and `rest` are the same operations on lists, with readable names (the names [2] uses). Both work; pick one style and stay with it.

1.3.2 Lists Are Nested Pairs

A list is not a separate kind of thing. It is a chain of pairs in which the right component is always another list, ending in `empty` (also written `'()`):

Definition 3 (List)

A list is either `empty`, or `(cons x l)` where `x` is any value and `l` is a list.

```
(define l (cons 1 (cons 2 (cons 3 (cons 4 empty)))))
(define l-short (list 1 2 3 4))
```

`l` and `l-short` are the same value; `list` is shorthand for the chain of `cons`es (Figure 1.3). Both print as `'(1 2 3 4)`. Then `(first l)` is `1` and `(rest l)` is `'(2 3 4)`.

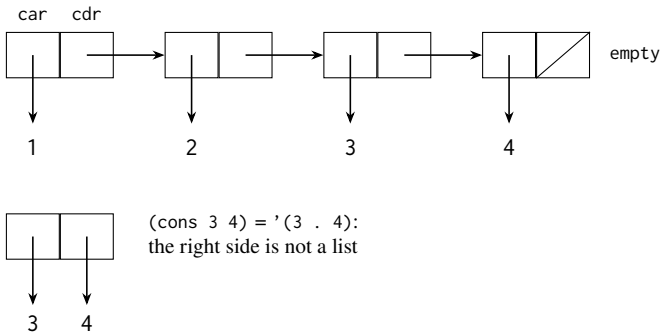


Figure 1.3: Box-and-pointer diagrams. Top: `'(1 2 3 4)`, four pairs whose `cdr`s chain to `empty` (drawn as a slash). Bottom: a bare pair.

Pitfall 4 (The dot)

The dot in `'(3 . 4)` means the right side of that pair is not a list, so the chain does not end in `empty`. Whenever a dot appears in output where you expected a list, you built a bare pair somewhere.

The two-case definition is why list functions keep having two `cond` clauses: one for `empty`, one for a `cons`.

1.3.3 Reversing a List

Key Point 5 (The shape of the function follows the shape of the data)

A list is empty or a `cons`, so a function on lists has those two cases. In the `cons` case, the recursive call is on the `rest`, the part of the data that is itself a list.

The empty list reversed is the empty list. Otherwise, reverse everything after the first element, then put the first element at the end:

```
(define (my-reverse l)
  (cond
    [(empty? l) empty]
    [else (append (my-reverse (rest l)) (list (first l)))]))
```

`append` joins two lists, so both arguments must be lists. That is why the first element is wrapped as `(list (first l))`, the one-element list. Rewriting by hand:

```
(my-reverse '(1 2 3))
---> (append (my-reverse '(2 3)) '(1))
---> (append (append (my-reverse '(3)) '(2)) '(1))
---> (append (append '(3) '(2)) '(1))
---> (append '(3 2) '(1))
---> '(3 2 1)
```

This works, but `append` itself walks its whole left argument, and it is called once per element, so `my-reverse` takes $O(n^2)$ time. Racket's built-in `reverse` uses an accumulator instead, the same trick as `fact-tail`; Recitation 1 writes that version (Section 1.4).

1.3.4 (SUPPLEMENT) Further Topics

Cost model of lists. Racket lists are singly linked and immutable. `cons`, `first`, `rest`, and `empty?` take constant time. `length`, `list-ref`, and `append` walk the list: `(append a b)` copies every pair of `a` and shares `b` unchanged. This is why building a list at the front with `cons` and reversing once at the end is the standard idiom.

Proper and improper lists. Racket's `list?` checks the whole chain and is true only for proper lists, while `pair?` looks at one cell:

```
> (pair? '(3 . 4))
#t
> (list? '(3 . 4))
#f
> (append '(3) 4)
'(3 . 4)
```

The last one surprises people: `append` does not check that its *last* argument is a list, so it can produce an improper list. Recitation 1's warm-up asks about exactly these cases.

1.4 Recitation 1: Functional Programming in Racket

The recitation practices the basics of the lecture in DrRacket, on a running example about traffic lights. The handout is itself a program: it is written in `#lang scribble/lp2`, a literate-programming language in which code sits in named `@chunk` blocks inside the prose. "Scribble HTML/PDF" renders the document; "Run" runs every chunk listed in the top-level `<*>` chunk. Tests are written with `check-expect` from `test-engine/racket-tests`, and the final `(test)` reports the results.

1.4.1 Expressions and Functions

```
(define (is-even x)
  (= 0 (modulo x 2)))
```

Exercise 6 (Predict at the REPL)

What do `is-even`, `(is-even)`, `(is-even 2)`, `(is-even 3)`, `(is-even #t)`, and `(is-even 'four)` evaluate to?

Solution. `is-even` alone is the function value, printed `#<procedure:is-even>`; a function is a value like any other. `(is-even)` is an *arity mismatch*: the function expects one argument. `(is-even 2)` is `#t` and `(is-even 3)` is `#f`. `(is-even #t)` and `(is-even 'four)` are *contract violations* raised inside `modulo`, which expects integers. Racket checks this only when the call runs: nothing stops you from writing the call.

Building programs by composition. In functional style, programs are built by composing functions. A traffic light is one of `'red`, `'yellow`, `'green`. Two functions are given:

- `can-go` EXPECTS a traffic light and PRODUCES a boolean: `#t` when a vehicle approaching the light may safely enter the intersection.
- `next-light` EXPECTS a traffic light and PRODUCES the light that comes after it.

```
(define (can-go l) (equal? l 'green))

(define (next-light l)
  (match l
    ['red    'green]
    ['yellow 'red]
    ['green  'yellow]))
```

`match` compares `l` against each pattern in turn; a quoted symbol pattern matches that symbol. (Section 3.4 looks at `match` in detail.)

Exercise 7 (Pedal to the metal)

Write `can-floor-it`: it EXPECTS a traffic light and PRODUCES `#t` when a vehicle can safely enter the intersection at either the current light or the next one.

Solution. Compose the two given functions:

```
(define (can-floor-it l)
```

```
  (or (can-go l) (can-go (next-light l))))
(check-expect (can-floor-it 'green) #t)
(check-expect (can-floor-it 'red) #t)
(check-expect (can-floor-it 'yellow) #f)
```

Red is safe because the next light is green; yellow is not, because the next light is red.

1.4.2 Specifications

Before writing a function, state which inputs are valid and what output to expect. The handout's format is a list of EXPECTS and PRODUCES lines plus one sentence of meaning. For `next-light-iter`:

- EXPECTS a non-negative integer `n` and a traffic light `l`;
- PRODUCES a traffic light: the color of `l` after it changes `n` times.

The specification answers the two questions the code needs. When `n` is zero, the light has not changed, so the answer is `l`. When `n` is positive, the answer is the light after changing $n - 1$ times starting from `(next-light l)`.

```
(define (next-light-iter n l)
  (if (eq? n 0)
      l
      (next-light-iter (- n 1) (next-light l))))

(check-expect (next-light-iter 0 'green) 'green)
(check-expect (next-light-iter 1 'green) 'yellow)
(check-expect (next-light-iter 2 'yellow) 'green)
```

The tests come straight from the specification: one for the base case, one for a single step, and one that wraps around the cycle (yellow, red, green). The recursive call is a tail call, with the light itself playing the role of an accumulator.

1.4.3 Recursion: Scrolling Through Lights

A vehicle is one of `'sports-car`, `'suv`, `'prius`. `speed` gives its speed in meters per second and `time-to-change` gives how many seconds a light stays on:

```
(define (speed v)
  (match v
    ['sports-car 136]
    ['suv        48]
    ['prius      27]))

(define (time-to-change l)
  (match l
    ['red    10]
    ['yellow 2]
    ['green  23]))
```

Exercise 8 (Scroll)

`safe-to-scroll` EXPECTS a vehicle `v`, a non-negative distance `d` in meters, and a light `l` that has just turned on. It PRODUCES `#t` if the vehicle, driving at constant speed, enters the intersection without a violation. Hint: write a helper that decides whether the vehicle enters before the light changes.

Solution. If the vehicle reaches the intersection before the light changes, the answer is whether *this* light allows entry. Otherwise the problem shrinks: the vehicle has driven for one full light, so it is closer, and the light is now the next one.

```
(define (will-enter v d l)
  (<= d (* (time-to-change l) (speed v))))

(define (safe-to-scroll v d l)
  (if (will-enter v d l)
      (can-go l)
      (safe-to-scroll
       v
       (- d (* (time-to-change l) (speed v)))
       (next-light l))))

(check-expect (safe-to-scroll 'sports-car 137 'green) #t)
```

Two traces show both outcomes. A Prius covers $23 \times 27 = 621$ m during a green light:

```
(safe-to-scroll 'prius 1000 'green)    ; 1000 > 23*27 = 621
---> (safe-to-scroll 'prius 379 'yellow)  ; 379 > 2*27 = 54
---> (safe-to-scroll 'prius 325 'red)     ; 325 > 10*27 = 270
---> (safe-to-scroll 'prius 55 'green)    ; 55 <= 621: enters on green
---> #t

(safe-to-scroll 'suv 1200 'green)      ; 1200 > 23*48 = 1104
---> (safe-to-scroll 'suv 96 'yellow)     ; 96 <= 2*48 = 96: enters on yellow
---> #f
```

The recursion terminates because every recursive call has a strictly smaller `d`: each light lasts a positive time and every vehicle has positive speed.

Remark 9. The handout's prose writes the call as `(safe-to-scroll v l d)`, while the posted code takes `(v d l)`. The code and its test are consistent, so the order is `(v d l)`. This is the kind of mismatch a specification is meant to catch.

1.4.4 Lists and Higher-Order Programming

Warm-up: `cons` vs. `append`.

expression	result
`(cons 3 4)`	`'(3 . 4)`, a bare pair
`(cons 3 '(4))`	`'(3 4)`
`(cons '(4) 3)`	`'((4) . 3)`: a pair whose `car` is a list
`(append 3 4)`	error: `append: contract violation`
`(append 3 '(4))`	error: the first argument must be a list
`(append '(3) 4)`	`'(3 . 4)`: the last argument is not checked

`cons` puts one element in front of a list; `append` joins lists. Neither complains about every misuse, so watch for dots in the output.

Exercise 10 (All even)

Write `all-even-rec`: it EXPECTS a list of integers and PRODUCES `#t` when every number in it is even.

Solution. The TA solution uses `match` with a `cons` pattern, which tests for a non-empty list and names its two parts in one step:

```
(define (all-even-rec l)
  (match l
    ['()              #t]
    [(cons head tail) (and (is-even head) (all-even-rec tail))]))
(check-expect (all-even-rec '()) #t)
(check-expect (all-even-rec '(2 4 6)) #t)
(check-expect (all-even-rec '(2 4 6 7)) #f)
```

The empty list is all even (there is no counterexample), which is also what makes `#t` the right base value for `and`.

Exercise 11 (Map)

Given `all`, which checks that a list of booleans is all `#t`, write `all-even-map` without explicit recursion.

```
(define (all l) (andmap identity l))
```

Solution. The intended answer is `(all (map is-even l))`: `map` turns the numbers into booleans and `all` combines them. The TA solution skips the intermediate list and passes `is-even` to `andmap` directly:

```
(define (all-even-map l) (andmap is-even l))
```

`(andmap f l)` is `#t` when `f` returns true on every element. It stops at the first element that fails, whereas `map` would first compute all the booleans.

Exercise 12 (Reverse, and reverse with a tail call)

Write a function that reverses a list. Then write a version that uses tail-call optimization.

Solution. The direct version is `my-reverse` from Section 1.3, written with `match`:

```
(define (reverse-list l)
```

```
  (match l
    ['() '()]
    [(cons head tail) (append (reverse-list tail) (list head))]))
```

Its calls pile up pending appends:

```
(reverse-list '(1 2 3 4 5 6 7))
---> (append (reverse-list '(2 3 4 5 6 7)) (list 1))
---> (append (append (reverse-list '(3 4 5 6 7)) (list 2)) (list 1))
---> ...
```

On a list with millions of elements, that is a call stack millions deep. The tail-recursive version carries the reversed-so-far list in an accumulator. Moving elements one at a time from the front of `l` to the front of `acc` reverses their order, and `cons` is the right tool, since it adds to the front:

```
(define (reverse-helper l acc)
  (match l
    ['() acc]
    [(cons head tail) (reverse-helper tail (cons head acc))]))

(define (reverse-list-tail l)
  (reverse-helper l '()))
```

```
(reverse-helper '(1 2 3) '())
---> (reverse-helper '(2 3) '(1))
---> (reverse-helper '(3) '(2 1))
---> (reverse-helper '() '(3 2 1))
---> '(3 2 1)
```

This is also faster: $O(n)$ instead of $O(n^2)$, since there is no `append`.

Remark 13 (Tail-call *elimination*)**.** When a language *guarantees* the optimization, it is often called tail-call elimination. Racket guarantees it; many languages, including Rust and Java, do not.

1.4.5 (SUPPLEMENT) Further Topics

`eq?` vs. `equal?`. The handout's margin asks for the difference. `equal?` compares structure: two lists with equal elements are `equal?`. `eq?` compares identity: are these the very same object in memory? Symbols are *interned*, so two occurrences of `'red` are the same object and `eq?` works on them, which is why `can-go` could use either. Lists built separately are different objects:

```
> (eq? 'red 'red)
#t
> (eq? (list 1) (list 1))
#f
> (equal? (list 1) (list 1))
#t
```

`eqv?` sits in between: like `eq?`, but also compares numbers and characters by value. Use `equal?` unless you specifically mean identity. (`next-light-iter` uses `eq?` on numbers; `=` or `zero?` would say what is meant more clearly.)

The design recipe. The EXPECTS/PRODUCES format is a light version of the design recipe of [2]: (1) a data definition, (2) a signature and purpose statement, (3) examples, written as tests, (4) a template that follows the data definition (one case per variant, a recursive call per self-reference), (5) the body, and (6) running the tests. Most of the recursive functions in this course are written by step 4 alone.

Chapter 2

Week 2: Higher-Order Functions, Scope, and State

Lecture 2: September 15, 2026. Recitation 2: September 18, 2026.

Supplementary reading:

- [4] Sections 4.4–4.6 (functions, local binding), 4.8 (sequencing), 5 (programmer-defined datatypes), and 12 (pattern matching).
- [2] Part II (self-referential data) and Part III (abstraction: `map`, `filter`, folds).
- [3] Sections 1.3 (higher-order procedures) and 3.1 (assignment and local state).

Lecture 2 finishes the tour of Racket: more list recursion and accumulators, structs for tree-shaped data, functions as values, scope, and finally state. Recitation 2 then writes the first interpreter of the course, for Boolean expressions, together with an optimization pass. This chapter covers:

- Section 2.1: adding to the end of a list, a quadratic `reverse` and a broken one, and accumulators packaged as named `let`;
- Section 2.2: `struct`, binary trees, and `match`;
- Section 2.3: `lambda`, closures, `map`, `filter`, and the two folds;
- Section 2.4: `let` and `let*`, lexical scope, pure code and side effects, and boxes;
- Section 2.5: Recitation 2, an interpreter and a De Morgan optimizer for Boolean expressions, and how to test an optimization.

2.1 Lists and Accumulators

Throughout this chapter we use the names `empty?`, `first`, and `rest` rather than `null?`, `car`, and `cdr`. They mean the same thing.

2.1.1 snoc: Adding to the End

`(cons x l)` makes one new pair whose head is `x` and whose tail is the old list `l`. Nothing is copied, so adding to the *front* costs $O(1)$ however long the list is.

Adding to the *back* is different. The old list ends in `'()`, and that `'()` cannot be replaced without rebuilding every pair on the way to it. So a back-insert walks the whole list, in $O(n)$. Racket does not provide one; we write it and call it `snoc`, "cons" backwards:

```
(define (snoc l x)
  (cond
    [(empty? l) (list x)]
    [else (cons (first l) (snoc (rest l) x))]))
```

Two shapes of input, two branches: the skeleton of almost every list function in this course. The empty case answers "the list containing just `x`"; the other keeps the first element in place and hands the rest of the problem to `snoc`. So `(snoc '() 'x)` is `'(x)` and `(snoc '(1 2) 'x)` is `'(1 2 x)`.

2.1.2 Reversing, Correctly and Incorrectly

To reverse a list, reverse everything after the first element, then put the first element at the very end, which is exactly `snoc`:

```
(define (rev l)
  (cond
    [(empty? l) '()]
    [else (snoc (rev (rest l)) (first l))]))
```

```
(rev '(1 2 3))
---> (snoc (rev '(2 3)) 1)
---> (snoc (snoc (rev '(3)) 2) 1)
---> (snoc (snoc '(3) 2) 1)
---> (snoc '(3 2) 1)
---> '(3 2 1)
```

This is correct but slow: one `snoc` per element, each $O(n)$, so $O(n^2)$ in total. It also needs $O(n)$ stack, because the `snoc` is wrapped *around* the recursive call, which therefore is not a tail call.

Here is a tempting rewrite with `cons` instead of `snoc`:

```
(define (rev-alt l)
  (cond
    [(empty? l) '()]
    [else (cons (rev-alt (rest l)) (first l))]))
```

```
> (rev-alt '(1 2 3))
'((() . 3) . 2) . 1)
```

Pitfall 14 (cons is not symmetric)

`cons` expects an element on the left and a list on the right. `rev-alt` passes them the other way round, so it builds a nest of pairs, not a list (Figure 2.1). There is no error, because a pair may hold any two values: the wrong argument order is a *silent* bug, and the dots in the output are the only sign. Swapping the arguments does not rescue this approach: `cons` can only ever add to the front, and reversing needs to add to the back.

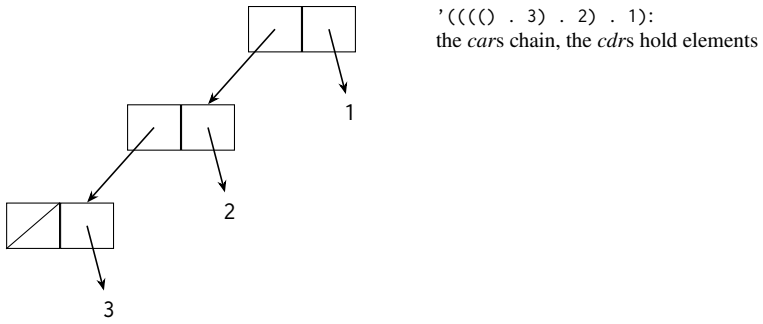


Figure 2.1: What `rev-alt` builds: every pair has a non-list on its right, hence the dots.

2.1.3 Aggregating a List

Summing a list follows the same skeleton:

```
(define (sum-list l)
  (cond
    [(empty? l) 0]
    [else (+ (first l) (sum-list (rest l)))]))
```

The `+` sits *outside* the recursive call. It cannot happen until the recursion returns, so every level must be remembered, and a long enough list runs out of stack.

As with `fact` in Section 1.2, the fix is to carry the running total as an extra argument, making the recursive call a tail call:

```
(define (sum-list-helper l acc)
  (cond
    [(empty? l) acc]
    [else (sum-list-helper (rest l) (+ (first l) acc))]))

(define (sum-list-acc l)
  (sum-list-helper l 0))
```

```
(sum-list-helper '(1 2 3) 0)
---> (sum-list-helper '(2 3) 1)
---> (sum-list-helper '(3) 3)
---> (sum-list-helper '() 6)
---> 6
```

2.1.4 Named `let`: Hiding the Helper

A top-level helper with an extra argument is noise. A *named* `let` packages the same thing as a loop: the name after `let` becomes a local function, and the bindings give its parameters and their starting values.

```
(define (sum-list-loop l)
  (let loop ([l l]
             [acc 0])
    (cond
      [(empty? l) acc]
      [else (loop (rest l) (+ (first l) acc))])))
```

This is the idiomatic way to write a loop in Racket; it runs exactly like the helper version.

2.1.5 (SUPPLEMENT) Further Topics

What a named `let` means. A named `let` is shorthand for defining a local recursive function and calling it immediately:

```
(let loop ([l l] [acc 0]) body)
;; means
((letrec ([loop (lambda (l acc) body)]) loop) l 0)
```

`letrec` (Section 2.4) is needed because `loop` refers to itself.

When the accumulator changes the answer. Turning `sum-list` into `sum-list-acc` is safe because `+` is associative and commutative: the accumulator adds the elements in the opposite grouping, $((0 + 1) + 2) + 3$ instead of $1 + (2 + (3 + 0))$. Do the same to `snoc`-based list building and the result comes out reversed, which is exactly how `reverse-helper` in Section 1.4 works. Section 2.3 makes the two groupings precise as `foldl` and `foldr`.

2.2 Structs and Trees

2.2.1 `struct`

`struct` defines a new kind of compound value. One line

```
(struct bt (label left right) #:transparent)
```

generates five functions:

- `bt`, the constructor: `(bt 1 empty empty)`;
- `bt?`, a predicate that answers `#t` for these values;
- `bt-label`, `bt-left`, `bt-right`, the accessors for the three fields.

Pitfall 15 (Forgetting `#:transparent`)

Without `#:transparent`, a struct value prints as `#<bt>` with its contents hidden, and `equal?` is true only when the two values are the very same object. Two trees that look identical then compare as different, and every test that compares trees fails. With the flag, values print readably and `equal?` compares field by field.

`struct` does *not* check the types of its fields. `(bt 1 2 3)` is accepted, even though 2 and 3 are not trees. Nothing goes wrong until some traversal calls `bt-label` on the number 2, possibly far from the line that built the bad value. Section 3.2 returns to this point: `define-type` in `plai-typed` rejects such a value where it is built.

2.2.2 Binary Trees

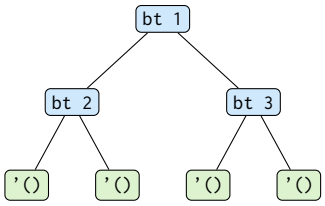
The empty tree is represented by `'()`, so a binary tree is either `empty` or a `bt` with a label and two subtrees.

```
(define tree
  (bt 1
      (bt 2 empty empty)
      (bt 3 empty empty)))
```

The definition of a tree mentions trees in *two* places, left and right, so a function over trees has two recursive calls. Lists mentioned themselves once, and list functions had one. The shape of the code follows the shape of the data (Figure 2.2).

```
(define (sum-bt t)
  (cond
    [(empty? t) 0]
    [(bt? t) (+ (bt-label t)
                (sum-bt (bt-left t))
                (sum-bt (bt-right t)))]
    [else (error 'sum-bt "not a tree: ~a" t)]))
```

Tree recursion is usually not made tail-recursive: with two recursive calls, at most one of them can be the last thing the function does, and the other has to be remembered.



(sum-bt tree)
$= 1 +$ (sum-bt (bt 2 ...)) $+$ (sum-bt (bt 3 ...))
$= 1 + (2 + 0 + 0) + (3 + 0 + 0)$
$= 6$

Figure 2.2: The value `tree`. Each `bt` node causes two recursive calls, one per subtree.

2.2.3 `match`

`match` checks the shape and extracts the fields in one step, which reads better once all the fields are needed anyway:

```
(define (sum-bt/match t)
  (match t
    ['() 0]
    [(bt label left right)
     (+ label (sum-bt/match left) (sum-bt/match right))]))
```

The pattern `(bt label left right)` matches only `bt` values and binds the three names to the fields in order.

2.2.4 (SUPPLEMENT) Further Topics

Structural recursion. Functions like `sum-bt` are *structurally recursive*: they recur only on immediate sub-parts of their input. Such functions always terminate on finite data, since each call is on a strictly smaller value, and their correctness can be proved by *structural induction*, one case per variant. Every interpreter in this course is structurally recursive in its `eval` on the abstract syntax tree, except where substitution builds a new tree (Chapter 4).

Opaque structs and abstraction. Opacity is the default for a reason: an opaque struct hides its representation, so a module can export the struct without letting clients inspect or compare its insides. Transparent structs are the right choice for plain data such as trees and syntax, where structural equality is what you want. A middle ground is `#:prefab`, which also makes structs readable and writable as literals.

Contracts on fields. Plain Racket can check field types at run time with `struct/contract` or `contract-out`, e.g. requiring `left` to satisfy `(or/c empty? bt?)`. The error then points at the bad construction rather than the later traversal. `plai-typed` (Chapter 3) instead checks before the program runs.

2.3 Higher-Order Functions

2.3.1 Functions Are Values

A function does not need a name. `lambda` builds one on the spot, and it can be called immediately: `((lambda (x) (* x x)) 4)` is `16`. So `define` for functions is shorthand, and these two definitions are the same:

```
(define (square x)
  (* x x))

(define square-alt
  (lambda (x) (* x x)))
```

Use the short form. Do not nest one form inside the other: `(define (f x) (lambda ...))` defines a function that *returns* a function, which is a different thing.

Since functions are values, a function can return one:

```
(define (make-adder x)
  (lambda (y) (+ x y)))

(define add2 (make-adder 2))
```

Definition 16 (Closure)

A *closure* is a function value together with the bindings of the variables it refers to from the surrounding code.

`(make-adder 2)` returns a function that has not been called yet but already remembers that `x` is 2. Then `(add2 5)` is 7, and `((make-adder 3) 4)` is 7 as well.

2.3.2 `map` and `filter`

Once functions can be arguments, the two most common list patterns become ordinary functions. `map` applies a function to every element; the result has the same length as the input.

```
(define (my-map f l)
  (cond
    [(empty? l) '()]
    [else (cons (f (first l)) (my-map f (rest l)))]))
```

So

```
(map f (list a b c))                    ; = (list (f a) (f b) (f c))
(map (lambda (x) (+ 1 x)) (list 1 2 3))    ; = '(2 3 4)
```

`filter` keeps the elements that pass a test; the contents are unchanged, only the length shrinks. To drop an element, return the filtered rest without consing it back on:

```
(define (my-filter f l)
  (cond
    [(empty? l) '()]
    [else (if (f (first l))
```

```
            (cons (first l) (my-filter f (rest l)))
            (my-filter f (rest l)))]))
```

`(filter odd? (list 1 2 3 4 5))` is `'(1 3 5)`.

Key Point 17 (Three shapes of list code)

`map`: list to list, same length. `filter`: list to list, shorter. `fold`: list to a single value. Most list code is a combination of these three.

2.3.3 Folds

`map` and `filter` each fix what happens at every step. A *fold* takes the step as an argument: a two-argument `combine`, a `base` value for the empty list, and the list. It walks the list once and collapses it into one value. There are two folds, and they differ in which way the parentheses group (Figure 2.3):

$$\texttt{(my-foldl - 0 '(1 2 3))} = ((0-1)-2)-3 = -6,$$
$$\texttt{(my-foldr - 0 '(1 2 3))} = 1-(2-(3-0)) = 2.$$

With an associative and commutative `combine` such as `+`, `*`, or `max`, the two agree, which is why the difference is easy to miss until it bites.



`foldl`: $((0-1)-2)-3$ `foldr`: $1-(2-(3-0))$

Figure 2.3: The two folds of `-` over `'(1 2 3)` with base 0 (green). `foldl` leans left, starting from the base; `foldr` leans right, ending at the base.

`foldl` combines on the way down. In `my-foldl`, `base` doubles as the running accumulator. Each step folds one more element into it and calls itself, and that call is a tail call, so this is a loop in constant stack space:

```
(define (my-foldl combine base l)
  (cond
    [(empty? l) base]
    [else (my-foldl combine (combine base (first l)) (rest l))]))
```

```
(my-foldl + 0 '(1 2 3 4 5))
---> (my-foldl + 1 '(2 3 4 5))
---> (my-foldl + (+ 1 2) '(3 4 5))
---> (my-foldl + (+ 3 3) '(4 5))
---> (my-foldl + (+ 6 4) '(5))
---> (my-foldl + 15 empty)
---> 15
```

Nothing is pending; at every step the accumulator is a value and the list is shorter.

`foldr` combines on the way back up. In `my-foldr`, `combine` wraps the recursive call. No arithmetic can happen until the recursion reaches `'()` and the base value comes back:

```
(define (my-foldr combine base l)
  (cond
    [(empty? l) base]
    [else (combine (first l) (my-foldr combine base (rest l)))]))
```

```
(my-foldr + 0 '(1 2 3 4 5))
---> (+ 1 (my-foldr + 0 '(2 3 4 5)))
---> (+ 1 (+ 2 (my-foldr + 0 '(3 4 5))))
---> (+ 1 (+ 2 (+ 3 (+ 4 (+ 5 (my-foldr + 0 empty))))))
---> (+ 1 (+ 2 (+ 3 (+ 4 (+ 5 0)))))
---> 15
```

That growing pile of pending additions *is* the stack: it is the concrete reason `foldr` uses $O(n)$ space and `foldl` $O(1)$.

Key Point 18 (`foldr` rebuilds structure)

`(my-foldr cons '() l)` copies `l`, because `foldr` replaces each `cons` of the list by `combine` and the final `'()` by `base`. Hence `map` and `filter` are both folds in disguise:

```
(define (map-as-fold f l)
  (my-foldr (lambda (x acc) (cons (f x) acc)) '() l))

(define (filter-as-fold f l)
  (my-foldr (lambda (x acc) (if (f x) (cons x acc) acc)) '() l))
```

Pitfall 19 (Racket's `foldl` argument order)

The `combine` of `my-foldl` takes `(acc elem)`. Racket's built-in `foldl` calls `(combine elem acc)`, element first, the opposite order. So `(foldl - 0 '(1 2 3))` in Racket is $3 - (2 - (1 - 0)) = 2$, not -6. Check the order before reusing a `combine`.

2.3.4 (SUPPLEMENT) Further Topics

Currying. `make-adder` is a *curried* version of `+`: instead of taking two arguments at once it takes one and returns a function waiting for the other. Any two-argument function can be curried this way. Section 3.1.3 shows the difference in the types, and Recitation 4 uses a curried helper so it can be passed to `map` (Section 4.5).

The universality of `foldr`. Every structurally recursive function on lists that only uses the head and the result for the tail can be written as a `foldr`: `length`, `append`, `map`, `filter`, `reverse` (inefficiently), and more. In typed functional languages this is the starting point for program calculation: laws such as `foldr` $f\ b \circ$ `map` $g =$ `foldr` $(\lambda x\ a.\ f\ (g\ x)\ a)\ b$ let a compiler fuse two traversals into one.

Closures need the environment. How does `add2` remember `x`? An implementation stores, alongside the code of the `lambda`, the bindings of its free variables at the time the `lambda` was evaluated. Implementing this is a central topic of the interpreter part of the course: Chapter 4 first implements functions by substitution, and [1, Chapters 6–7] replace substitution by environments and closures.

2.4 Scope and State

2.4.1 `let` and `let*`

`let` binds names to values for the extent of its body. The clauses go in one pair of brackets; square and round brackets are interchangeable in Racket, so both of these are the same form:

```
(let ([x (+ 1 2)])
  (+ x x))           ; 6

(let [(x (+ 1 2))
      (y (+ 4 5))]
  (+ x y))           ; 12
```

The catch is *when* each name starts to exist. `let` evaluates all of its right-hand sides first, in the scope *outside* the `let`, and only then binds the names. So a later clause cannot use an earlier one:

```
(let ([x (+ 1 2)]
      [y (+ 1 x)])     ; x is not bound here
  (+ x y))
;; x: undefined; cannot reference an identifier before its definition
```

Read the message as "that name is not in scope here".

`let*` binds the clauses one at a time, each in the scope of the previous ones, so this version works and gives 7:

```
(define let-example
  (let* ([x (+ 1 2)]
         [y (+ 1 x)])
    (+ x y)))
```

There is no magic in `let*`; it is nested `let`s:

`(let* ([x a] [y b]) body)` means `(let ([x a]) (let ([y b]) body))`.

Use `let` by default, since it tells the reader the bindings are independent, and `let*` only when a clause really needs the one above it. Neither lets a clause refer to *itself*: a local recursive function needs `letrec`, or the named `let` of Section 2.1.

2.4.2 Lexical Scope

A `let` is an expression, so it can appear anywhere a value can, including in another `let`'s right-hand side:

```
(define nested-let-example
  (let ([x (let ([y 3]) (+ y y))])
    (+ x x)))          ; x = 6, so 12
```

But `y` exists only inside the inner `let`. Writing `(+ x y)` in the outer body is the same undefined-identifier error, even though `y` was "used" a moment earlier (Figure 2.4).

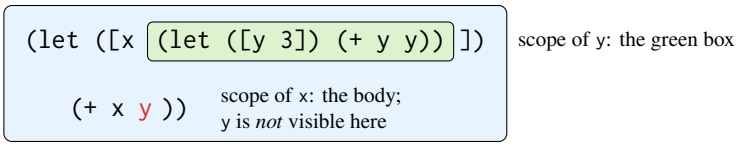


Figure 2.4: Scopes nest like the text. The body of the outer `let` is outside the inner `let`, so `y` is unbound there.

Definition 20 (Lexical scope)

Under *lexical* (static) scope, which binding a use of a name refers to is determined by where the binding is *written* in the program text, not by what happened to run before.

This is the same rule that makes the closure in `make-adder` work: the `x` in `(lambda (y) (+ x y))` refers to the `x` of the enclosing `define`, wherever the returned function is later called.

2.4.3 Pure Code and Side Effects

Everything so far is *pure*: a call depends only on its arguments and does nothing but return a value. Run the same expression twice and you get the same answer, so an expression can always be replaced by its value without changing the program. That is what makes the step-by-step traces of `rev` and the folds valid reasoning, not just illustration.

A *side effect* is anything else a call does: printing, mutating a variable, reading input, raising an error. In a language built on statements, `i++` is exactly this. Running it twice is not the same as running it once, so it cannot be replaced by its value.

- `"Hello World"` is a value. Evaluating it does nothing else.
- `(print "Hello World")` writes to the console and returns `(void)`. The output is the point; the return value is not.

Racket does not forbid effects; it makes them ordinary expressions, so staying pure is a choice made function by function.

Sequencing. A function body may hold several expressions. They run in order, the earlier ones only for their effects, and the value of the *last* one is returned:

```
(define (foo x)
  (print "Hello")
  (+ x 1))
```

`(foo 1)` returns 2 and prints along the way. The returned number is perfectly predictable; the printing is what makes `foo` impure. `begin` is the same sequencing as a single expression, for places where only one expression is allowed, such as a `cond` branch or an arm of `if`:

```
(begin
  (print "Hello")
  (print "World")
  (+ 1 2))          ; returns 3, after printing
```

Three ways to print. All return `(void)`; they differ only in what appears on the console. `(print "Hello")` shows `"Hello"`, the value as Racket would write it, quotes included. `(display "Hello")` shows `Hello`, the

human-readable form. `(printf "The number foo is: ~a !" (* 3 3))` shows `The number foo is: 9 !`: `~a` splices in the display form of the next argument.

2.4.4 Boxes: State You Can See

A *box* is a mutable container with exactly one slot, the smallest possible piece of state. It is a value like any other, so it can be passed to a function that changes it.

```
(define bar (box 3))

(define (increment-box b)
  (set-box! b (+ 1 (unbox b))))
```

`(box 3)` makes a box, `(unbox bar)` reads it, and `(set-box! bar 10)` overwrites it and returns `(void)`. `increment-box` is called for its effect and returns nothing useful, so the caller has to `unbox` afterwards to see the new contents:

```
(let ([b (box 3)])
  (increment-box b)
  (unbox b))          ; 4
```

The point of a box is that mutability is *visible*. A plain 3 can never change; a box holding 3 announces that someone may change it, so a reader can tell where the state lives. By convention, a name ending in `!` mutates something: a warning that calling it twice is not the same as calling it once.

2.4.5 (SUPPLEMENT) Further Topics

`letrec`. `letrec` makes the new names visible in all the right-hand sides, so they can refer to themselves and each other:

```
(letrec ([even? (lambda (n) (if (= n 0) #t (odd? (- n 1))))]
         [odd?  (lambda (n) (if (= n 0) #f (even? (- n 1))))])
  (even? 10))         ; #t
```

Internal `defines` in a function body behave like `letrec`. Implementing recursion in an interpreter is surprisingly subtle; [1, Chapter 9] shows that it needs either mutation or a fixed-point trick.

Dynamic scope. The alternative to lexical scope is *dynamic* scope, where a name refers to its most recent binding *at run time*. Under dynamic scope,

```
(define (f) x)
(let ([x 5]) (f))
```

would return 5, because `x` was bound to 5 when `f` ran; under lexical scope it is an error, since no `x` is visible where `f` is written. Early Lisps and Emacs Lisp (by default) used dynamic scope. It makes a function's meaning depend on every caller, which is why almost all modern languages are lexically scoped. Racket offers a controlled form of dynamic binding through *parameters* (`make-parameter`, `parameterize`).

Boxes vs. `set!`. Racket also has `set!`, which mutates a variable directly: `(set! x (+ x 1))`. Boxes make the mutable thing a first-class value; `set!` makes every variable potentially mutable, which complicates reasoning (and closures that capture a mutated variable see the change). [1, Chapter 8] implements both and

explains why they need different machinery in an interpreter: a *store* that maps locations to values, threaded through every evaluation step.

Referential transparency. "An expression can be replaced by its value" is called *referential transparency*. It is what licenses equational reasoning and compiler optimizations such as common-subexpression elimination. Languages such as Haskell enforce it everywhere and track effects in types; Racket leaves it to discipline.

2.5 Recitation 2: Boolean Expressions

This recitation builds the course's first interpreter and first program transformation, for a tiny language of Boolean expressions. Everything is in plain Racket, with structs and `match`.

2.5.1 The Language and Its Representation

The grammar of *bexp*:

$$\langle bexp \rangle \ ::= \ \langle boolean \rangle \ \mid \ \langle bexp \rangle \wedge \langle bexp \rangle \ \mid \ \langle bexp \rangle \vee \langle bexp \rangle \ \mid \ \neg \langle bexp \rangle$$

One struct per alternative of the grammar represents it in Racket:

```
(struct leaf-node (arg))
(struct or-node (left right))
(struct and-node (left right))
(struct not-node (arg))

(define bexp-bool-1 (and-node (leaf-node #t) (leaf-node #f)))
(define bexp-sym-1 (and-node (leaf-node 'x) (leaf-node 'y)))
```

A value built from these structs is an *abstract syntax tree*: the tree structure replaces parentheses and precedence. Note that the structs are *not* transparent and their fields are unchecked: `bexp-sym-1` has symbols, not booleans, at its leaves, and Racket accepts it.

Exercise 21 (Syntactic equality)

Write `syn-equal?`, which checks whether two bexps are the same tree.

Solution. Because the structs are opaque, `equal?` would compare them by identity (Section 2.2), so we write structural equality ourselves. `match*` matches several values at once, one pattern per value:

```
(define (syn-equal? b1 b2)
  (match* (b1 b2)
    [((and-node l1 r1) (and-node l2 r2)) (and (syn-equal? l1 l2) (syn-equal? r1 r2))]
    [((or-node l1 r1) (or-node l2 r2))   (and (syn-equal? l1 l2) (syn-equal? r1 r2))]
    [((not-node a1) (not-node a2))       (syn-equal? a1 a2)]
    [((leaf-node v1) (leaf-node v2))     (equal? v1 v2)]
    [(_ _) #f]))
```

One clause per variant where both sides agree, and a final catch-all for mismatched shapes.

Pitfall 22 (A missing variant hides behind a catch-all)

If the `not-node` clause were left out, two `not-nodes` would fall through to `[(_ _) #f]`, and `syn-equal?` would say a negation is not equal to itself. Nothing would complain, and every later test comparing expressions with a negation would fail even with a correct optimizer. With a catch-all clause present, check the list of variants, not just the clauses you were given.

2.5.2 An Interpreter for bexp

Exercise 23 (Interpreter)

Write `evaluate-bexp`, which computes the Boolean value an expression represents.

Solution. One clause per variant, and a recursive call wherever a variant contains a sub-expression:

```
(define (evaluate-bexp b)
  (match b
    [(leaf-node v)    v]
    [(or-node b1 b2)  (or (evaluate-bexp b1) (evaluate-bexp b2))]
    [(and-node b1 b2) (and (evaluate-bexp b1) (evaluate-bexp b2))]
    [(not-node b1)    (not (evaluate-bexp b1))]))

(check-expect (evaluate-bexp (leaf-node #t)) #t)
(check-expect (evaluate-bexp (and-node (leaf-node #t) (leaf-node #f))) #f)
(check-expect (evaluate-bexp (and-node (leaf-node #f) (leaf-node #t))) #f)
```

The third test swaps the arguments of the second; testing both orders is cheap and catches an implementation that looks at only one side.

This is the pattern of every interpreter in the course: our language's $\wedge$ is implemented by Racket's `and`, its $\vee$ by `or`, its $\neg$ by `not`. The *meaning* of each construct is given by the clause that handles it.

Missing cases. What happens with an incomplete `match`?

```
(define (evaluate-bexp-m b)
  (match b
    [(leaf-node v) v]))
```

Nothing complains when it is defined. The error appears only when it is called on a value that no clause matches: `(evaluate-bexp-m bexp-bool-1)` raises `match: no matching clause`. Racket checks patterns at run time, so a missing case is found only if some test happens to reach it. Section 3.4 redoes this in `plai-typed`, where the same mistake is rejected before the program runs.

2.5.3 An Optimization Pass: De Morgan

De Morgan's laws and double-negation elimination state the equivalences

$$\neg(P \wedge Q) = \neg P \vee \neg Q, \qquad \neg(P \vee Q) = \neg P \wedge \neg Q, \qquad \neg\neg P = P.$$

Using them left to right, an optimizer can eliminate redundant negations and push every remaining negation down to the leaves. The examples use symbolic leaves:

name	before	after
`bexp-1`	$\neg(\neg P \vee Q)$	$P \wedge \neg Q$
`bexp-2`	$\neg\neg P \vee \neg Q$	$P \vee \neg Q$
`bexp-3`	$P \vee \neg Q$	$P \vee \neg Q$ (already optimal)

```
(define bexp-1 (not-node (or-node (not-node (leaf-node 'P)) (leaf-node 'Q))))
(define bexp-2 (or-node (not-node (not-node (leaf-node 'P))) (not-node (leaf-node 'Q))))
(define bexp-3 (or-node (leaf-node 'P) (not-node (leaf-node 'Q))))

(define bexp-1-opt (and-node (leaf-node 'P) (not-node (leaf-node 'Q))))
(define bexp-2-opt (or-node (leaf-node 'P) (not-node (leaf-node 'Q))))
(define bexp-3-opt (or-node (leaf-node 'P) (not-node (leaf-node 'Q))))
```

What does it mean for an optimization to be correct?. Before writing the optimizer, decide how to check it. An optimization is correct when it does two things:

1. it *preserves semantics*: the output evaluates to the same value as the input;
2. it actually *optimizes*: the output has the promised shape, here every negation sitting directly on a leaf.

Both become executable checks:

```
(define (preserves-sem? unopt)
  (equal? (evaluate-bexp unopt) (evaluate-bexp (demorgan-opt unopt))))

(define (is-optimizing? opt)
  (define (only-not-at-leaves? bexp)
    (match bexp
      [(leaf-node v) #t]
      [(or-node b1 b2) (and (only-not-at-leaves? b1) (only-not-at-leaves? b2))]
      [(and-node b1 b2) (and (only-not-at-leaves? b1) (only-not-at-leaves? b2))]
      [(not-node (leaf-node v)) #t]
      [(not-node _) #f]))
  (only-not-at-leaves? opt))

(define (is-correct? unopt)
  (and (preserves-sem? unopt) (is-optimizing? (demorgan-opt unopt))))
```

`is-optimizing?` relies on `match` trying clauses in order: `(not-node (leaf-node v))` is tried first, so the catch-all `(not-node _)` sees only negations of compound expressions, which mean the pass left work undone. Note also that `preserves-sem?` takes only the input and calls `demorgan-opt` itself, so it tests the optimizer rather than a hand-written answer.

The optimizer. What to do with a negation depends entirely on what is *inside* it, so a single `(not-node b1)` clause cannot work. Nested patterns split it into the four shapes the argument can have, one per variant. This is what the handout means by "you will need more cases than in `evaluate-bexp`":

```
(define (demorgan-opt b)
  (match b
    ; Standard Stuff
    [(leaf-node v)    (leaf-node v)]
    [(or-node b1 b2)  (or-node (demorgan-opt b1) (demorgan-opt b2))]
    [(and-node b1 b2) (and-node (demorgan-opt b1) (demorgan-opt b2))]
    [(not-node (leaf-node l)) (not-node (leaf-node l))]
    ; De-Morgan Stuff
    [(not-node (and-node b1 b2))
     (or-node (demorgan-opt (not-node b1)) (demorgan-opt (not-node b2)))]
```

```
[(not-node (or-node b1 b2))
 (and-node (demorgan-opt (not-node b1)) (demorgan-opt (not-node b2)))]
[(not-node (not-node b1)) (demorgan-opt b1)]))
```

- **A negated leaf** is already as far down as a negation can go; keep it.
- **De Morgan.** After rewriting $\neg(b_1 \wedge b_2)$ to $\neg b_1 \vee \neg b_2$, the new negations may themselves sit on top of an `and`, an `or`, or a `not`, so they are optimized again: the recursive calls are on `(not-node b1)`, not on `b1`. Calling `(demorgan-opt b1)` would drop the negation and change the meaning.
- **Double negation.** The two negations cancel, and the result is `(demorgan-opt b1)`, not just `b1`, because `b1` may still contain negations to push down.

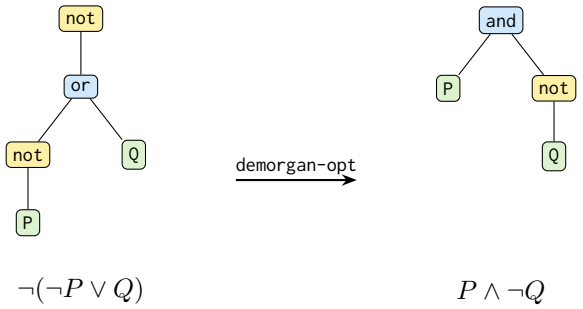


Figure 2.5: `bexp-1` before and after the pass. Negations (yellow) end up only directly above leaves.

For `bexp-1` (Figure 2.5):

```
(demorgan-opt (not (or (not P) Q)))
---> (and-node (demorgan-opt (not (not P))) (demorgan-opt (not Q)))   ; not over or
---> (and-node (demorgan-opt P) (not Q))                              ; not-not cancels
---> (and-node P (not Q))
```

Why it terminates. Every recursive call is on a smaller tree. In the De Morgan cases the argument `(not-node b1)` is new, but it is smaller than the input `(not-node (and-node b1 b2))`, which has one more node and all of `b2` besides.

2.5.4 Testing the Optimizer Two Ways

The syntactic tests compare against the exact trees written by hand:

```
(check-expect (syn-equal? (demorgan-opt bexp-1) bexp-1-opt) #t)
(check-expect (syn-equal? (demorgan-opt bexp-2) bexp-2-opt) #t)
(check-expect (syn-equal? (demorgan-opt bexp-3) bexp-3-opt) #t)
```

The semantic tests check the two correctness properties without writing the expected tree at all:

```
(define bexp-4 (not-node (or-node (not-node (leaf-node #t)) (leaf-node #f))))
(define bexp-5 (or-node (not-node (not-node (leaf-node #t))) (not-node (leaf-node #f))))
(define bexp-6 (or-node (leaf-node #t) (not-node (leaf-node #t))))
(check-expect (is-correct? bexp-4) #t)
(check-expect (is-correct? bexp-5) #t)
(check-expect (is-correct? bexp-6) #t)
```

Remark 24 (Why booleans at the leaves)**.** `bexp-4` to `bexp-6` are `bexp-1` to `bexp-3` with `#t` and `#f` in place of `'P` and `'Q`. With symbols at the leaves, `evaluate-bexp` is meaningless: `(not 'P)` is `#f` and `(or 'P #f)` is `'P`, because Racket treats every value except `#f` as true. So a semantic check only means something on Boolean leaves.

Remark 25 (The handout's table)**.** An earlier version of the handout listed $\neg(\neg P \wedge Q) \mapsto P \wedge Q$ as the first example. That contradicts both the code (`bexp-1` is built with `or-node`) and De Morgan's law; the posted solution corrects the row to $\neg(\neg P \vee Q) \mapsto P \wedge \neg Q$. Trust the definitions over the prose, and check the prose by working one example by hand.

2.5.5 (SUPPLEMENT) Further Topics

Negation normal form. The output shape of `demorgan-opt`, with negations only on variables, is called *negation normal form* (NNF). It is the first step in converting a formula to conjunctive normal form (CNF), the input format of SAT solvers. NNF conversion never increases the number of $\wedge$/$\vee$ nodes, but the further step to CNF can blow up exponentially without tricks such as the Tseitin transformation.

Testing semantics exhaustively. `preserves-sem?` checks one assignment of truth values per test. For symbolic expressions, a stronger test enumerates every assignment to the variables (there are 2^n for n variables) and compares the two expressions under each, which decides equivalence completely for small n. Random generation of expressions plus such a check is *property-based testing*, available in Racket through the `rackcheck` package and in Haskell as QuickCheck.

Short-circuiting is inherited. `evaluate-bexp` implements $\wedge$ with Racket's `and`, which does not evaluate its second argument when the first is `#f`. Our language therefore short-circuits too, without anyone deciding it should. For a pure language like bexp this is invisible, but once the language has errors or effects it matters, and an interpreter that wants strict $\wedge$ must evaluate both sides first, e.g. `(let ([v1 ...] [v2 ...]) (and v1 v2))`. This is the inheritance problem of definitional interpreters (Section 1.1).

Chapter 3

Week 3: Types and a First Interpreter

Lecture 3: September 22, 2026. Recitation 3: September 25, 2026.

Supplementary reading:

- [1] Chapters 2 (Everything (We Will Say) About Parsing) and 3 (A First Look at Interpretation).
- [1] Chapter 15 (Checking Program Invariants Statically: Types), first sections, and the `plai-typed` documentation.
- [5] Chapter 1 (what type systems are for), optional.

From this week on, code is written in `#lang plai-typed`, the language of the textbook: Racket with a type checker that reads the whole program before anything runs. Lecture 3 introduces its types, defines new types with `define-type`, and uses them to write the first interpreter of the course, a calculator. Recitation 3 redoes Recitation 2 with types and then studies Racket's `match` in more depth. This chapter covers:

- Section 3.1: types, type inference and annotations, function types, homogeneous lists, pairs, and polymorphism;
- Section 3.2: `define-type`, `type-case`, and exhaustiveness checking;
- Section 3.3: quote and s-expressions, concrete vs. abstract syntax, and the calculator's parser and interpreter;
- Section 3.4: Recitation 3, typed Boolean expressions and pattern matching with `match`.

3.1 Types in plai-typed

`plai-typed` is installed with `raco pkg install plai-typed`, and a file starts with `#lang plai-typed`. It looks like Racket, but a type checker reads the whole program first, and a program that fails the check does not run at all.

3.1.1 Every Value Has a Type

At the REPL, every answer comes with two lines: the type the checker worked out, starting with `-`, and then the value.

```
> 3
- number
3
> (+ 3 4)
- number
7
> "hello"
- string
"hello"
> (+ 3 "hello")
typecheck failed: number vs. string
```

This is the idea of the lecture in miniature: the type is known *before* the value is computed. The checker does not run `(+ 3 4)` to find out it is a number; it knows because `+` takes numbers and returns a number.

3.1.2 Type Inference and Annotations

No types need to be written. Here the checker sees `(+ 3 x)`, concludes that `x` must be a number, and that the result is one too:

```
(define (add3 x)
  (+ 3 x))
```

Definition 26 (Type inference)

Type inference is the checker working out the types that the programmer did not write down.

Types can also be written, as many or as few as we like. An argument becomes `(x : number)`, and the return type goes after the argument list. Whatever is left out is inferred:

```
(define (add3-v2 (x : number))            ; return type inferred
  (+ 3 x))

(define (add3-v3 (x : number)) : number   ; both annotated
  (+ 3 x))
```

Pitfall 27 (Spaces around the colon)

`(x: number)` is read as one identifier named `x:`, followed by `number`, and is rejected with `expected either <id> or '[<id> : <type>]'`. Write `(x : number)`.

An annotation is a claim the checker verifies, not a hint it trusts. This one is wrong, since `(+ 3 x)` is a number, so the definition is rejected:

```
(define (add3-v4 (x : number)) : string
  (+ 3 x))
;; typecheck failed: number vs. string
```

Either way the program is checked before it runs: a call like `(add3-v3 "hello")` never gets the chance to fail at run time, because the whole file is rejected.

If the checker can infer the types, why write them? Two reasons from lecture:

- **Documentation.** The reader sees what the function takes and returns without reading its body.
- **Mental model.** Writing the type first makes you decide what the function is supposed to do before you write how it does it.

3.1.3 Function Types

Functions are values, so they have types. Read the arrow as "takes . . . and returns . . . ":

```
> add3
- (number -> number)
```

With two arguments, both go to the left of the arrow. Annotations can be mixed: in `myadd-v2`, `y` is unannotated but added to a number, so it is inferred to be one, and the two functions have the same type.

```
(define (myadd x y)
  (+ x y))
(define (myadd-v2 (x : number) y) : number
  (+ x y))
```

```
> myadd
- (number number -> number)
> myadd-v2
- (number number -> number)
```

A function that returns a function, `make-adder` from Section 2.3, now has a type that says so:

```
(define (myadd-v3 (x : number))
  (lambda (y) (+ x y)))
```

```
> myadd-v3
- (number -> (number -> number))
> (myadd-v3 2)
- (number -> number)
> ((myadd-v3 2) 5)
- number
7
```

Compare the two types. `myadd` takes two numbers at once. `myadd-v3` takes *one* number and returns a function that waits for the second, so `(myadd-v3 2)` is not a number yet but a function.

3.1.4 Lists Are Homogeneous

A list type says what its elements are, and every element must have that type:

```
> (list 1 2 3 4)
- (listof number)
'(1 2 3 4)
> (cons 3 (cons 4 empty))
- (listof number)
'(3 4)
> (list 1 "hello")
typecheck failed: number vs. string
> (cons 3 4)
typecheck failed: (listof number) vs. number
```

Plain Racket accepts both of the last two. In `plai-typed` the second argument of `cons` must be a list, so bare pairs (Section 1.3) are impossible. In exchange, list operations have precise types: the checker knows `(first (list 1 2 3 4))` is a `number` and `(rest (list 1 2 3 4))` a `(listof number)`, and likewise for `second` and `third`, without running anything.

3.1.5 Pairs

When two values of different types belong together, use a *pair* (a tuple). Its type is written with `*`:

```
> (pair 1 "hello")
- (number * string)
'#(1 "hello")
> (fst (pair 1 2))
- number
1
> (snd (pair 1 2))
- number
2
> (car (pair 1 2))
car: unbound identifier
```

A list has any length but one element type; a pair has exactly two slots, each with its own type. A pair is not a cons cell, and `plai-typed` provides no `car` or `cdr` at all: use `fst`/`snd` for pairs and `first`/`rest` for lists.

3.1.6 Polymorphic Types

Asking for the type of `first` shows something new:

```
> first
- ((listof 'a) -> 'a)
> rest
- ((listof 'a) -> (listof 'a))
> cons
- ('a (listof 'a) -> (listof 'a))
> fst
- (('a * 'b) -> 'a)
> map
- (('a -> 'b) (listof 'a) -> (listof 'b))
```

Definition 28 (Type variable, parametric polymorphism)

`'a` (read "alpha") is a *type variable*: any type may stand in its place, but every `'a` in one type must stand for the *same* type. A single definition that works uniformly at every type in this way is *parametrically polymorphic*; other languages call it generics.

So `first` takes a `(listof number)` to a `number` and a `(listof string)` to a `string`, but can never take a list of numbers to a string. Different letters allow different types: `fst` takes a pair of an `'a` and a `'b` and returns the `'a`. And the type of `map` alone says most of what `map` does: given a function from `'a` to `'b` and a list of `'a`, it produces a list of `'b`.

3.1.7 Why Insist on Homogeneous Lists?

Suppose `(list 1 "hello")` were allowed; call it `l`. What is the type of `(first l)`? It depends on which element happens to come first, and the checker cannot know that before the program runs. Then

- every use of `first` would need a check at run time, which is exactly what a type checker is supposed to spare us, and
- `(map f l)` is only safe if `f` accepts *both* numbers and strings, which `('a -> 'b)` cannot express.

One could design a type that lists every position, something like `(listof number string number)` for `(list 1 "hello" 2)`, but then every list function (`map`, `filter`, the folds) would need a far more complicated type. The simpler fix keeps lists homogeneous and instead makes *one* type that covers all the kinds of things to be mixed. That is what `define-type` is for.

3.1.8 (SUPPLEMENT) Further Topics

How inference works. `plai-typed` uses Hindley–Milner style inference, the algorithm behind ML, OCaml, and Haskell. Every unannotated position gets a fresh type variable; every use generates an equation (`(+ 3 x)` demands $\tau_x =$ `number`); and *unification* solves the equations. If they have no solution, the error names the two types that clashed, as in `number vs. string`. Whatever remains unconstrained becomes a type variable such as `'a`, which is how `first` gets a polymorphic type without any annotation.

Soundness. The guarantee behind "rejected before it runs" is *type soundness*: a well-typed program never reaches a state where an operation is applied to the wrong kind of value. It is usually proved as *progress* (a well-typed expression is a value or can take a step) plus *preservation* (a step keeps the type) [5, Chapter 8]. Soundness does not rule out every error: `(first empty)` and division by zero still fail at run time, because the type `(listof number)` does not say whether the list is empty.

`plai-typed` vs. Typed Racket. Racket ships another typed language, Typed Racket, designed to type existing Racket programs. It supports union types such as `(U Number String)`, so `(list 1 "hello")` is fine there, with type `(Listof (U Number String))`, and occurrence typing that refines a union after a test like `number?`. `plai-typed` is deliberately simpler, which is what makes full inference possible.

3.2 define-type and type-case

3.2.1 Defining a Type with Variants

A shape is either a circle or a rectangle:

```
(define-type Shape
  [circle (radius : number)]                       ; variant 1
  [rectangle (width : number) (height : number)])  ; variant 2
```

This defines one new type, Shape, with two *variants*. Each variant has a constructor and typed fields:

```
> (circle 3)
- Shape
(circle 3)
```

Now `(list (circle 1) (rectangle 2 3))` is fine: the elements are different kinds of shapes, but they have one type, so the list is a `(listof Shape)`.

Like `struct`, `define-type` also generates

- predicates `circle?` and `rectangle?`, of type `(Shape -> boolean)`, and
- accessors `circle-radius`, `rectangle-width`, and `rectangle-height`, of type `(Shape -> number)`.

The predicates accept only shapes. In plain Racket `(circle? 5)` would return `#f`; here it never runs:

```
> (circle? 5)
typecheck failed: Shape vs. number
```

A recursive type. A type may mention itself. The binary tree of Section 2.2, now with types:

```
(define-type Tree
  [emp]
  [node (n : number) (lt : Tree) (rt : Tree)])
```

A variant with no fields is still a constructor, so the empty tree is `(emp)`, with parentheses. `(node 3 (emp) (emp))` has type Tree.

3.2.2 struct vs. define-type

Both make compound values, but they promise different things:

`struct` (`#lang racket`)	`define-type` (`plai-typed`)
one kind of value: a `bt` is always a `bt`	one *type* with several variants: `circle` or `rectangle`
fields unchecked: `(bt 1 2 3)` is accepted, and fails later	every field typed: `(node 3 5 (emp))` is rejected before the program runs, at the line that is wrong
different structs cannot share a homogeneous list type	all variants fit in one `(listof Shape)`

Why only one `circle?`. No other type in the program may also have a variant named `circle`. The immediate reason is that `circle`, `circle?`, and `circle-radius` are ordinary definitions, so a second `define-type` would define them again (`identifier already defined`). The deeper reason is inference: when the checker sees

(circle 3), the constructor's name is all it has to go on. If two types both had a circle variant, (circle 3) could belong to either, and inference would be stuck.

3.2.3 type-case: Taking a Value Apart

A define-type value is built with a constructor; type-case takes it apart again. There is one clause per variant, and the names in parentheses are bound to that variant's fields, in order (the names need not match those in define-type; only the order matters):

```
(define (area s)
  (type-case Shape s
    [circle (r) (* 3.14159 (* r r))]
    [rectangle (w h) (* w h)]))
```

```
> (area (circle 3))
- number
28.27431
> area
- (Shape -> number)
```

The same function can be written with the generated predicates and accessors:

```
(define (area-v2 s)
  (cond
    [(circle? s) (* 3.14159 (* (circle-radius s) (circle-radius s)))]
    [(rectangle? s) (* (rectangle-width s) (rectangle-height s))]))
```

It works, but type-case is shorter, names the fields, and lines the code up with the type: one clause per variant, just as list functions had one cond branch per kind of list.

3.2.4 Exhaustiveness

The real difference shows when a case is forgotten. type-case knows every variant of Shape, so leaving one out is rejected before the program runs:

```
(define (area-v3 s)
  (type-case Shape s
    [circle (r) (* 3.14159 (* r r))]))
;; type-case: syntax error; probable cause: you did not include a case
;; for the rectangle variant, or no else-branch was present
```

cond knows nothing about Shape; it tries its tests in order. The same mistake with cond is accepted, works on circles, and fails only at run time when it meets a rectangle:

```
(define (area-v4 s)
  (cond
    [(circle? s) (* 3.14159 (* (circle-radius s) (circle-radius s)))]))
```

```
> (area-v4 (circle 1))
- number
3.14159
> (area-v4 (rectangle 2 3))
cond: no matching clause
```

Key Point 29 (What `type-case` buys)

`type-case` checks before the program runs that every variant is handled, and it gives each clause the variant's fields with their types. A missing case is a static error, not a bug waiting for the right input.

3.2.5 (SUPPLEMENT) Further Topics

Algebraic data types. A `define-type` is an *algebraic data type*: a *sum* (one of several variants) of *products* (each variant a tuple of fields). The names come from counting: if a type has $|A|$ values, then $A \times B$ has $|A| \cdot |B|$ values and $A + B$ has $|A| + |B|$. The same idea appears as `type ... = | ...` in OCaml, `data` in Haskell, `enum` with fields in Rust and Swift, and sealed interfaces with records in Java 21. All of them check `match` expressions for exhaustiveness, as `type-case` does.

Why exhaustiveness is checkable. Exhaustiveness is decidable because the set of variants is *closed*: `define-type` lists all of them in one place, and no other code can add one. Object-oriented class hierarchies are *open*, since any file can add a subclass, so a compiler cannot check that a chain of `instanceof` tests is complete. This is one side of the "expression problem": with variants, adding a new function is easy and adding a new variant forces you to revisit every function, which the checker then helpfully enumerates.

The `else` clause. `type-case` also accepts a final `[else ...]`, which makes any set of clauses exhaustive. It is convenient but defeats the check: when a variant is added later, the compiler will not point at this `type-case`. Interpreters in this course avoid `else` in `type-case` for that reason.

3.3 S-Expressions and the Calculator

3.3.1 Quote and Symbols

The apostrophe `'`, read "quote", means "do not evaluate this; keep it as data". Without it, `x` is a variable to look up. With it, `'x` is the name itself, a *symbol*:

```
> x
x: free variable while typechecking in: x
> 'x
- symbol
'x
```

Quoting a parenthesized expression keeps the whole thing as data, without doing the addition:

```
> '(1 (+ 2 3) 4)
- s-expression
'(1 (+ 2 3) 4)
```

3.3.2 S-Expressions

Look at what `'(+ 2 3)` contains: a symbol and two numbers. That is exactly the kind of mixed list that Section 3.1 ruled out. So `plai-typed` gives all quoted data one type, `s-expression`, and provides functions to ask what a piece is and to convert it back out:

```
> (s-exp->list '(+ 2 3))
- (listof s-expression)
'(+ 2 3)
> (first (s-exp->list '(+ 2 3)))
- s-expression
'+
> (s-exp->symbol (first (s-exp->list '(+ 2 3))))
- symbol
'+
> (s-exp->number (second (s-exp->list '(+ 2 3))))
- number
2
```

Key Point 30 (Ask, then convert)

Ask with `s-exp-number?`, `s-exp-symbol?`, or `s-exp-list?`, then convert with the matching `s-exp->number`, `s-exp->symbol`, or `s-exp->list`. A conversion fails at run time if the guess was wrong.

Pitfall 31 (A lone quoted symbol is not an s-expression)

`'x` has type `symbol`, not `s-expression`, so `(parse 'x)` is a type error. To get an s-expression for one symbol, write `` `x `` (backquote) or `(symbol->s-exp 'x)`.

3.3.3 Concrete and Abstract Syntax

The first interpreter is a calculator:

```
'(+ 2 3)                    ===> 5
'(* 3 4)                    ===> 12
'(* (+ 2 3) (* 5 (+ 6 7)))  ===> 325
```

Following [1, Chapters 2–3], the work is split into separate steps (Figure 3.1):

- **Concrete syntax** is what the programmer types, the text `(+ 2 3)`. Racket's *reader* already turns text into an s-expression, so we get that step for free; quote is how we hand an s-expression over directly.
- **Abstract syntax** is a tree that keeps only the meaning. `parse` turns an s-expression into this tree, rejecting anything that is not a valid program.
- The **interpreter** walks the tree and computes the answer. [1] calls it `interp`; in lecture it is `eval`.

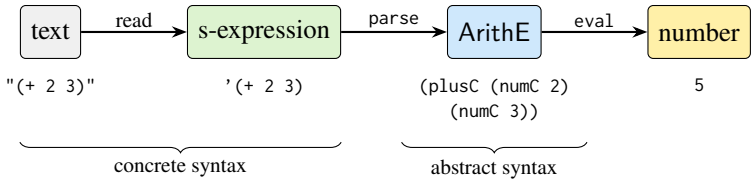


Figure 3.1: The interpreter pipeline. Racket's reader provides the first step.

3.3.4 The Calculator

Abstract syntax. One variant per kind of expression:

```
(define-type ArithE
  [numC (n : number)]
  [plusC (e1 : ArithE) (e2 : ArithE)]
  [timesC (e1 : ArithE) (e2 : ArithE)])
```

The operands of `plusC` are ArithE, not `number`. That is what allows `'(* (+ 2 3) 4)`: the left operand of the multiplication is itself an addition, the same trick as Tree having subtrees (Figure 3.2).

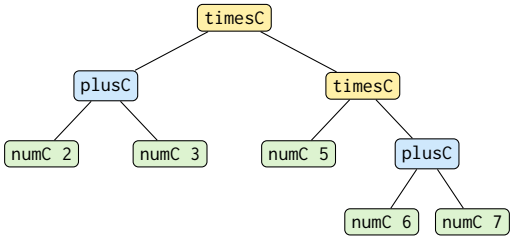


Figure 3.2: The abstract syntax of `'(* (+ 2 3) (* 5 (+ 6 7)))`. Parentheses are gone; the nesting is the tree.

The interpreter. One clause per variant:

```
(define (eval (e : ArithE)) : number
  (type-case ArithE e
    [numC (n) n]
    [plusC (e1 e2) (+ (eval e1) (eval e2))]
    [timesC (e1 e2) (* (eval e1) (eval e2))]))
```

Three variants, three clauses, and a recursive call wherever the type mentions ArithE. `eval` never checks whether its input makes sense, and it does not have to: a value of type ArithE can only be built from the three constructors, so every tree it receives is well formed.

The parser. That guarantee has to come from somewhere, and it comes from `parse`. The version written in lecture:

```
(define (parse (s : s-expression)) : ArithE
  (cond
    [(s-exp-number? s) (numC (s-exp->number s))]
    [(s-exp-list? s)
     (let [(l (s-exp->list s))]
       (case (s-exp->symbol (first l))
         [(+) (plusC (parse (second l)) (parse (third l)))]
         [(*) (timesC (parse (second l)) (parse (third l)))]))]))
```

A number becomes a `numC`. A list is dispatched on its first element with `case`, which compares a symbol against literal symbols, and each operand is parsed recursively. Putting the steps together, `(eval (parse '(* (+ 2 3) (* 5 (+ 6 7)))))` is 325.

3.3.5 What Malformed Input Does

The lecture parser is right on well-formed input, but on bad input it either crashes with a message about something else, or quietly builds a wrong tree:

input	result
`(parse '(- 1 2))`	`case: no matching clause`
`(parse '(+ 1))`	`third: list contains too few elements`
`(parse '(+ 1 2 3))`	`(plusC (numC 1) (numC 2))`: the 3 is silently dropped

The last one is the worst kind of bug: no error, just a program that means something other than what was written. A more careful parser checks the shape of each form and reports what was expected. This version is not from lecture, but every check in it addresses one row of the table:

```
(define (parse (s : s-expression)) : ArithE
  (cond
    [(s-exp-number? s) (numC (s-exp->number s))]
    [(s-exp-list? s)
     (let [(l (s-exp->list s))]
       (if (and (= (length l) 3) (s-exp-symbol? (first l)))
           (case (s-exp->symbol (first l))
             [(+) (plusC (parse (second l)) (parse (third l)))]
             [(*) (timesC (parse (second l)) (parse (third l)))]
             [else (error 'parse "unknown operator")])
           (error 'parse "expected (op e1 e2)")))]
    [else (error 'parse "expected a number or a list")]))
```

- The length check rejects `'(+ 1)` and `'(+ 1 2 3)`.
- `s-exp-symbol?` rejects `'(1 2 3)`, whose head is not an operator at all.
- The `else` of `case` rejects operators the language does not have, such as `-`.

- The final `else` rejects anything that is neither a number nor a list, such as a lone symbol.

3.3.6 (SUPPLEMENT) Further Topics

Why parsing is cheap here. Real languages need a lexer and a parser, often generated from a grammar, to get from text to a tree. Lisp-family languages put that work into one generic step: the reader turns any text into nested lists of atoms, and each language then only has to interpret those lists. [1, Chapter 2] uses this to skip parsing entirely. The flip side is that the s-expression `'(+ 1 2 3)` is perfectly well formed as data; deciding that it is not a valid *calculator* program is the job of `parse`.

Homoiconicity and quasiquote. Racket code is itself written as s-expressions, so programs can build programs as data. Quasiquote makes that convenient: if `x` is 5, then `` `(+ ,x 1) `` is the list `'(+ 5 1)`. Everything is quoted except the parts marked with a comma, which are evaluated and spliced in; `,@xs` splices in the elements of a whole list. Racket macros are functions from syntax to syntax built on the same idea, which is a large part of why Racket is good at implementing languages.

Desugaring. The calculator has no subtraction. One way to add it without touching `eval` is to translate it in the parser: `(- a b)` becomes `(plusC a (timesC (numC -1) b))`. Languages are usually defined as a small *core* interpreted directly plus a *surface* that desugars into it [1, Chapter 4]. The core stays small enough to reason about, and the surface stays convenient.

3.4 Recitation 3: Typed Boolean Expressions and Pattern Matching

Recitation 3 has two halves. Part A (handout 3a) redoes Recitation 2's interpreter and De Morgan optimizer in `plai-typed`, to see what the type checker buys. Part B (handout 3b) returns to plain Racket and studies `match`: literal patterns, wildcards, and taking lists apart.

3.4.1 Part A: Boolean Expressions, Typed

From four structs to one type. In Section 2.5, a Boolean expression was built from four unrelated structs; "a bexp" existed only in our heads, and no field had a type. With `define-type`, the four become variants of *one* type, and every field says what it holds:

```
(define-type bexp-b
  [leaf-node-b (b : boolean)]
  [and-node-b (b1 : bexp-b) (b2 : bexp-b)]
  [or-node-b (b1 : bexp-b) (b2 : bexp-b)]
  [not-node-b (b : bexp-b)])
```

Exercise 32 (Typed evaluator)

Implement `evaluate-bexp-b` with `type-case`.

Solution. `type-case` plays the role `match` played last week:

```
(define (evaluate-bexp-b [b : bexp-b]) : boolean
  (type-case bexp-b b
    [leaf-node-b (v) v]
    [and-node-b (b1 b2) (and (evaluate-bexp-b b1) (evaluate-bexp-b b2))]
    [or-node-b (b1 b2) (or (evaluate-bexp-b b1) (evaluate-bexp-b b2))]
    [not-node-b (b1) (not (evaluate-bexp-b b1))]))

(test (evaluate-bexp-b (leaf-node-b #t)) #t)
(test (evaluate-bexp-b (and-node-b (leaf-node-b #t) (leaf-node-b #f))) #f)
(test (evaluate-bexp-b (not-node-b (or-node-b (leaf-node-b #f) (leaf-node-b #f)))) #t)
```

`test` is `plai-typed`'s `check-expect`, and `(print-only-errors true)` at the top of the file makes passing tests silent. This is the same shape as `eval` for ArithE in Section 3.3.

What do the types buy us?.

(a) **Bad leaves are rejected before anything runs.** Last week's `bexp-sym-1` was

```
(and-node (leaf-node 'x) (leaf-node 'y))
```

and `evaluate-bexp` happily returned `'y`: Racket treats every value except `#f` as true, so `(and 'x 'y)` is `'y`. No error, just an answer of the wrong kind. The typed version

```
(evaluate-bexp-b (and-node-b (leaf-node-b 'x) (leaf-node-b 'y)))
```

is rejected (`typecheck failed: boolean vs. symbol`) before anything runs, and the error points at the leaf that is wrong, not at some later function that trips over it.

(b) **Missing cases are rejected too.** Last week's `evaluate-bexp-m` handled only leaves; it was accepted and failed only when called on something else. The typed version

```
(define (evaluate-bexp-b-m [b : bexp-b]) : boolean
  (type-case bexp-b b
    [leaf-node-b (v) v]))
```

stops the program from starting: `you did not include a case for the and-node-b variant`. This is the `area-v3`/`area-v4` contrast of Section 3.2 again.

De Morgan, typed. Now the leaves are variables:

```
(define-type bexp
  [leaf-node (s : symbol)]
  [and-node (b1 : bexp) (b2 : bexp)]
  [or-node (b1 : bexp) (b2 : bexp)]
  [not-node (b1 : bexp)])
```

Exercise 33 (Port `demorgan-opt` to `plai-typed`)

Rewrite the optimizer of Section 2.5 for the type bexp.

Solution. Last week's `match` could name a nested pattern directly, such as `(not-node (and-node b1 b2))`. A `type-case` clause only takes apart the *outermost* constructor, so the `not-node` clause needs a second `type-case`, on the thing being negated:

```
(define (demorgan-opt [b : bexp]) : bexp
  (type-case bexp b
    [leaf-node (v) (leaf-node v)]
    [and-node (b1 b2) (and-node (demorgan-opt b1) (demorgan-opt b2))]
    [or-node (b1 b2) (or-node (demorgan-opt b1) (demorgan-opt b2))]
    [not-node (b1)
      (type-case bexp b1
        [leaf-node (v) (not-node (leaf-node v))]
        [and-node (b2 b3) (or-node (demorgan-opt (not-node b2))
                                   (demorgan-opt (not-node b3)))]
        [or-node (b2 b3) (and-node (demorgan-opt (not-node b2))
                                   (demorgan-opt (not-node b3)))]
        [not-node (b2) (demorgan-opt b2)])]))
```

The inner clauses are the four nested patterns of last week, one per variant of the negated expression: a negated leaf stays; $\neg(b_2 \wedge b_3)$ becomes $\neg b_2 \vee \neg b_3$ with both new negations optimized again; $\neg(b_2 \vee b_3)$ symmetrically; and $\neg\neg b_2$ becomes the optimized b_2. Both `type-case`s are checked for exhaustiveness, so a forgotten shape of negation is now a static error.

The tests no longer need `syn-equal?`: `test` compares `define-type` values structurally.

```
(define bexp-1 (not-node (or-node (not-node (leaf-node 'P)) (leaf-node 'Q))))
(define bexp-2 (or-node (not-node (not-node (leaf-node 'P))) (not-node (leaf-node 'Q))))
(define bexp-3 (or-node (leaf-node 'P) (not-node (leaf-node 'Q))))

(test (demorgan-opt bexp-1) (and-node (leaf-node 'P) (not-node (leaf-node 'Q))))
```

```
(test (demorgan-opt bexp-2) (or-node (leaf-node 'P) (not-node (leaf-node 'Q))))
(test (demorgan-opt bexp-3) (or-node (leaf-node 'P) (not-node (leaf-node 'Q))))
```

3.4.2 Part B: Pattern Matching with `match`

So far `match` has only taken structs apart. It can also match literal values, ignore parts of the input with wildcards, and take lists apart.

Matching on values. A pattern can be a literal, which makes `match` a compact conditional:

```
(define (factorial x)
  (match x
    [0 1]
    [_ (* x (factorial (- x 1)))]))

(define (factorial-if x)
  (if (= x 0)
      1
      (* x (factorial-if (- x 1)))))
```

_ is the *wildcard*: it matches anything and binds nothing. Clauses are tried top to bottom, so _ goes last as the catch-all. With two cases `match` and `if` are about the same size; with more, nested `if`s pile up:

```
(define (fibonacci-if x)
  (if (= x 0)
      0
      (if (= x 1)
          1
          (+ (fibonacci-if (- x 1)) (fibonacci-if (- x 2))))))
```

Exercise 34 (Fibonacci with `match`)

Rewrite `fibonacci-if` using `match`.

Solution. One line per case, and the structure stays flat:

```
(define (fibonacci x)
  (match x
    [0 0]
    [1 1]
    [_ (+ (fibonacci (- x 1)) (fibonacci (- x 2)))]))

(check-expect (fibonacci 1) 1)
(check-expect (fibonacci 10) 55)
(check-expect (fibonacci 10) (fibonacci-if 10))
```

Exercise 35 (What is `else`?)

`[else ...]` often appears as the last clause of a `match`. Predict `(match 5 [else else])`.

Solution. It is `5`. Inside `match`, `else` is not a keyword but a *pattern variable*, like `x`: it matches anything and is bound to what it matched. So here `else` is bound to 5 and the clause returns it. That is why `[else ...]` happens to work as a catch-all; but it is an accident, and it quietly binds a variable named `else` in the body. Use `_` instead.

Destructuring lists. Two attempts at "the third element of a list":

```
(define (my-third x)
  (match x
    [(list a b c d) c]
    [_ x]))

(define (other-third lst)
  (match lst
    [(cons _ (cons _ (cons third _))) third]
    [_ (error "List too short")]))
```

Exercise 36 (Predict)

On which inputs do the two functions behave differently, and why?

Solution. `(list a b c d)` matches a list of *exactly* four elements. `(cons _ (cons _ (cons third _)))` matches any list of *at least* three: the last `_` matches whatever rest is left, including `'()`.

input	`my-third`	`other-third`
`(list 2 3)`	`'(2 3)`	error `"List too short"`
`0`	`0`	error
`(list '(2 3) 2 4 5 6 7)`	the whole list	`4`
`(list 1 2 3 4)`	`3`	`3`

```
(check-expect (my-third (list 2 3)) (list 2 3))
(check-expect (my-third 0) 0)
(check-expect (my-third (list '(2 3) 2 4 5 6 7)) (list '(2 3) 2 4 5 6 7))
(check-expect (my-third (list 1 2 3 4)) 3)

(check-error (other-third (list 2 3)) "List too short")
(check-error (other-third 0) "List too short")
(check-expect (other-third (list '(2 3) 2 4 5 6 7)) 4)
(check-expect (other-third (list 1 2 3 4)) 3)
```

`my-third` is right only on four-element lists, and when it is wrong it does not fail: the fallback `[_ x]` hands back its input, so the caller gets a list or a number where it expected the third element. That is the same kind of silent wrong answer as the calculator parser turning `'(+ 1 2 3)` into $1 + 2$ (Section 3.3). Raising an error, as `other-third` does, is better.

Exercise 37 (Fix `my-third`)

Make `my-third` return the third element of any list with at least three elements and raise an error otherwise, using a `list` pattern. Hint: `_ ...` matches zero or more elements.

Solution.

```
(define (my-third-fixed x)
  (match x
    [(list _ _ c _ ...) c]
    [_ (error "List too short")]))

(check-error (my-third-fixed (list 2 3)) "List too short")
(check-expect (my-third-fixed (list 1 2 3)) 3)
(check-expect (my-third-fixed (list '(2 3) 2 4 5 6 7)) 4)
```

This is equivalent to `other-third`'s nested `cons` pattern, and easier to read.

3.4.3 `match` vs. `type-case`

The two halves used two tools for taking values apart. Each is strong where the other is weak:

	`match`	`type-case`
language	`racket`	`plai-typed`
nested patterns	yes: `(not-node (and-node b1 b2))`	no: one level; nest another `type-case`
literals, wildcards, lists	yes	no
missing case	run-time error, only if reached	rejected before the program runs

`match` is more expressive. `type-case` is more limited, and in exchange it can check, before anything runs, that every variant is handled. Recitation 4 (Section 4.5) implements a small `match` in our own interpreter.

3.4.4 (SUPPLEMENT) Further Topics

More `match` patterns. Racket's `match` has many more pattern forms [4, Section 12]: `(? even? n)` matches when the predicate holds and binds `n`; `(and p1 p2)` and `(or p1 p2)` combine patterns; `(list-rest a b rest)` names the tail; `(app f p)` matches `p` against `(f v)`; and a clause can carry a guard with `#:when`. Repeating a pattern variable, as in `(list x x)`, requires the two parts to be `equal?`.

Exhaustiveness for richer patterns. Languages with both nested patterns and types, such as OCaml, Haskell, and Rust, get the best of both columns of the table: the compiler checks a set of nested patterns for exhaustiveness and also warns about *redundant* clauses that can never match (like a clause after `_`). The check is done by computing which values are not yet covered, clause by clause, and it is the same algorithm a compiler uses to turn a `match` into an efficient decision tree.

Chapter 4

Week 4: Names and Functions

Lecture 4: September 29, 2026. Recitation 4: October 2, 2026.

Supplementary reading:

- [1] Chapter 5 (Adding Functions to the Language), which implements functions by substitution as this week does, and Chapter 6 (From Substitution to Environments) for where the course goes next.
- [5] Sections 5.2–5.3 (the untyped lambda calculus: free variables, substitution, and α-conversion), optional.

Lecture 4 gives the calculator *names* and then *functions*. Both are implemented by substitution, and the lecture's real subject is how subtle substitution is: when to evaluate the thing being substituted, how inner bindings shadow outer ones, and how a careless substitution can *capture* a variable and change a program's meaning. Recitation 4 adds a `match` form to the same interpreter. This chapter covers:

- Section 4.1: `idC` and `letC`, eager substitution, and shadowing;
- Section 4.2: lazy substitution, its costs and benefits, and the variable-capture bug;
- Section 4.3: capture-avoiding substitution with fresh names from `gensym`;
- Section 4.4: `lambda` and application, a new `Value` type, and capture again;
- Section 4.5: Recitation 4, pattern matching in the interpreter.

4.1 Names: `let` and Substitution

4.1.1 The Plan

At the end of Chapter 3 the calculator had `numC`, `plusC`, and `timesC`. The first new form is `(let x e1 e2)`: compute `e1`, replace every `x` in `e2` with that value, then compute `e2`.

```
(let x 5 (+ x x))
---> (+ 5 5)                 replace every x in the body with 5
---> 10

(let x (+ 1 5) (+ x x))
---> (let x 6 (+ x x))       e1 is evaluated first
---> (+ 6 6)
---> 12
```

4.1.2 Two New Variants

```
(define-type ArithE
  [numC (n : number)]
  [idC (x : symbol)]
  [plusC (e1 : ArithE) (e2 : ArithE)]
  [timesC (e1 : ArithE) (e2 : ArithE)]
  [letC (x : symbol) (e1 : ArithE) (e2 : ArithE)])
```

(Section 4.4 adds two more variants for functions; from then on every function on ArithE has a case for them.)

What is `idC` for?. An `idC` is a *use* of a name, such as the `x` in `(+ x 1)`. The operands of `plusC` must be ArithEs, so a bare symbol `'x` cannot go there; `idC` wraps the symbol so a name can sit in the tree until substitution replaces it. It does for symbols what `numC` does for numbers.

A name appears in two roles, and only one of them is an `idC`:

```
;; (let x 5 (+ x 1))
(letC 'x (numC 5) (plusC (idC 'x) (numC 1)))
;;    ^ binding: a plain symbol    ^ use: an idC
```

The name right after `let` is being *bound*. It is a plain symbol field of `letC`, because it is never evaluated or replaced; it only tells substitution which name to look for. Symbols like `+` and `let` never become `idC`s either: `parse` consumes them to decide which variant to build. Writing `(plusC 'x (numC 1))` is a type error, since `'x` is a symbol and not an ArithE.

Parsing. A lone symbol becomes an `idC`. In `(let x e1 e2)` the `x` is taken as a plain symbol, while `e1` and `e2` are parsed recursively:

```
(define (parse (s : s-expression)) : ArithE
  (cond
    [(s-exp-number? s) (numC (s-exp->number s))]
    [(s-exp-symbol? s) (idC (s-exp->symbol s))]
    [(s-exp-list? s)
     (let [(l (s-exp->list s))]
       (case (s-exp->symbol (first l))
```

```
        [(+) (plusC (parse (second l)) (parse (third l)))]
        [(*) (timesC (parse (second l)) (parse (third l)))]
        [(let) (letC (s-exp->symbol (second l)) (parse (third l)) (parse (fourth l)))]))]))
```

As in lecture 3, there is no error checking: `(parse '(let x 1))` fails inside `fourth` with `list contains too few elements`.

4.1.3 Eager Substitution

`(subst-eager v x e)` replaces every `x` in `e` with the number `v`. Most variants just recurse into their children; the interesting cases are the ones that mention names.

```
(define (subst-eager (v : number) (x : symbol) (e : ArithE)) : ArithE
  (type-case ArithE e
    [numC (n) (numC n)]
    [idC (y) (if (equal? x y) (numC v) (idC y))]
    [plusC (e1 e2) (plusC (subst-eager v x e1) (subst-eager v x e2))]
    [timesC (e1 e2) (timesC (subst-eager v x e1) (subst-eager v x e2))]
    [letC (y e1 e2)
          (letC y
                (subst-eager v x e1)
                (if (equal? x y) e2 (subst-eager v x e2)))]))
```

Why `(numC v)` and `(idC y)`?. `subst-eager` returns an ArithE tree, but `v` is a number, so it is wrapped as `(numC v)` to go back into the tree. An `idC` for a different name is left alone: `(idC y)` rebuilds the same node, which is equivalent to returning `e`.

```
(subst-eager 5 'x (plusC (idC 'x) (idC 'y)))   ; = (plusC (numC 5) (idC 'y))
```

4.1.4 Shadowing

What is `(let x 5 (let x 6 (+ x x)))`: 12 or 10? The answer is 12. The inner `let` binds its own `x`, so inside its body `x` means 6: the inner `x` *shadows* the outer one (Figure 4.1).

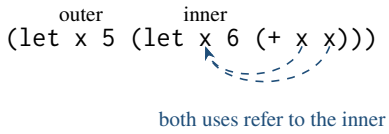


Figure 4.1: Each use of a name refers to the nearest enclosing binder of that name.

That is the `(equal? x y)` check in the `letC` case: when an inner `let` binds the same name we are substituting for, substitution stops at its body.

```
(let x 5 (let x 6 (+ x x)))
---> (let x 6 (+ x x))       the outer subst leaves the inner body alone
---> (+ 6 6)
---> 12
```

Substituting blindly would give `(let x 6 (+ 5 5))`, which is 10. The inner `e1` is still substituted, because it sits *outside* the inner binding; the inner `x` exists only in the inner body:

```
(let x 1 (let x (+ x 1) x))
---> (let x (+ 1 1) x)       the x in e1 is the outer x
---> 2
```

4.1.5 eval-eager

```
(define (eval-eager (e : ArithE)) : number
  (type-case ArithE e
    [numC (n) n]
    [idC (x) (error 'eval "unbound variable")]
    [plusC (e1 e2) (+ (eval-eager e1) (eval-eager e2))]
    [timesC (e1 e2) (* (eval-eager e1) (eval-eager e2))]
    [letC (y e1 e2) (eval-eager (subst-eager (eval-eager e1) y e2))]))
```

The `letC` line is the plan written out: evaluate `e1`, substitute the resulting number for `y` in `e2`, and evaluate the result. `eval-eager` never meets an `idC` that some `let` bound, because the enclosing `letC` has already substituted it away. An `idC` that survives was never bound, as in `(+ x 1)`, so it is an error.

```
(let x 5 (let y (+ x 1) (* x y)))
letC x: eval e1 = 5, subst 5 for x in the body
---> (let y (+ 5 1) (* 5 y))
letC y: eval e1 = 6, subst 6 for y in the body
---> (* 5 6)
---> 30
```

Each `let` removes the `idC`s for its own name; once every `let` has been processed, none is left.

4.1.6 (SUPPLEMENT) Further Topics

Free and bound variables, formally. The free variables of an expression are defined by structural recursion, with `let` as the only binder:

$$\mathrm{FV}(n) = \emptyset, \qquad \mathrm{FV}(x) = \{x\},$$
$$\mathrm{FV}(e_1 + e_2) = \mathrm{FV}(e_1 \times e_2) = \mathrm{FV}(e_1) \cup \mathrm{FV}(e_2), \quad \mathrm{FV}(\texttt{let}\ x\ e_1\ e_2) = \mathrm{FV}(e_1) \cup (\mathrm{FV}(e_2) \setminus \{x\}).$$

A closed expression ($\mathrm{FV}(e) = \emptyset$) never raises "unbound variable". Substitution $e[v/x]$ follows the same shape, and its `let` case is exactly the code above:

$$(\texttt{let}\ y\ e_1\ e_2)[v/x] = \begin{cases} \texttt{let}\ y\ e_1[v/x]\ e_2 & \text{if } y = x, \\ \texttt{let}\ y\ e_1[v/x]\ e_2[v/x] & \text{if } y \neq x. \end{cases}$$

Big-step semantics. The interpreter can be summarized by inference rules for a relation $e \Downarrow n$, "e evaluates to n". Each rule corresponds to one clause of `eval-eager`:

$$\frac{}{n \Downarrow n} \qquad \frac{e_1 \Downarrow n_1 \quad e_2 \Downarrow n_2}{e_1 + e_2 \Downarrow n_1 + n_2} \qquad \frac{e_1 \Downarrow n_1 \quad e_2[n_1/x] \Downarrow n}{\texttt{let}\ x\ e_1\ e_2 \Downarrow n}$$

There is no rule for a variable, which is why a free variable is an error: no derivation exists. Writing the rules side by side with the code is the mathematical approach to semantics mentioned in Section 1.1; for this language the two say exactly the same thing.

4.2 Lazy Substitution and Variable Capture

4.2.1 Lazy Substitution

An alternative: do not evaluate `e1` first. Substitute the unevaluated tree into the body, where it is evaluated each time the body uses it.

```
(let x (+ 1 1) (+ x 2))
eager:  ---> (let x 2 (+ x 2))  ---> (+ 2 2)  ---> 4
lazy:   ---> (+ (+ 1 1) 2)                    ---> 4
```

`subst-lazy` is `subst-eager` with `v` an ArithE, the unevaluated `e1`, which goes into the tree as is, with no `numC` wrapping:

```
(define (subst-lazy (v : ArithE) (x : symbol) (e : ArithE)) : ArithE
  (type-case ArithE e
    [numC (n) e]
    [idC (y) (if (equal? x y) v e)]
    [plusC (e1 e2) (plusC (subst-lazy v x e1) (subst-lazy v x e2))]
    [timesC (e1 e2) (timesC (subst-lazy v x e1) (subst-lazy v x e2))]
    [letC (y e1 e2)
          (letC y
                (subst-lazy v x e1)
                (if (equal? x y) e2 (subst-lazy v x e2)))]))

(define (eval-lazy (e : ArithE)) : number
  (type-case ArithE e
    [numC (n) n]
    [idC (x) (error 'eval "unbound variable")]
    [plusC (e1 e2) (+ (eval-lazy e1) (eval-lazy e2))]
    [timesC (e1 e2) (* (eval-lazy e1) (eval-lazy e2))]
    [letC (y e1 e2) (eval-lazy (subst-lazy e1 y e2))]))
```

Pitfall 38 (Mixing evaluators)

Every recursive call in `eval-lazy` must be to `eval-lazy`. If one called `eval-eager`, the type checker would not complain, since both return a number, but inner `lets` would silently become eager.

4.2.2 Is Lazy More Efficient?

When `x` is unused, yes: `e1` is never evaluated.

```
(let x (+ 5 6) 10)
eager:  (let x 11 10) ---> 10      (+ 5 6) computed once, for nothing
lazy:   10                         (+ 5 6) never computed
```

So `(let x (/ 5 0) 10)` would not be an error in the lazy version (if the language had `/`), and neither is an unbound name in `e1`: `(let x (+ y y) 10)` is an error eagerly but 10 lazily.

When `x` is used more than once, lazy is *less* efficient, since `e1` is recomputed at every use:

```
(let x (+ 5 6) (+ (+ x x) x))
eager:  (+ 5 6) once     ---> (+ (+ 11 11) 11)
lazy:   (+ 5 6) 3 times  ---> (+ (+ (+ 5 6) (+ 5 6)) (+ 5 6))
```

4.2.3 Where Eager and Lazy Differ

program	eager	lazy
`(let x (+ y y) 10)`	`unbound variable`	`10`
`(let y x (let x 2 y))`	`unbound variable`	`2`

The first difference is harmless: `x` is unused, so lazy never evaluates `e1` and never notices the unbound `y`. The second is a real bug.

4.2.4 Variable Capture

```
(let x y (let y 2 (+ x y)))
lazy:
---> (let y 2 (+ y y))        the free y is captured by the inner let
---> (+ 2 2)
---> 4
```

The outer `y` is free: nothing binds it, so the correct answer is an error. Racket agrees; the same program in `#lang racket` is rejected with `y: unbound identifier` before it runs. But lazy substitution moved that free `y` underneath `(let y 2 ...)`, where it now refers to the inner `y` (Figure 4.2).

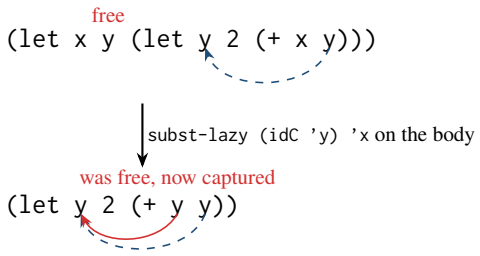


Figure 4.2: Capture. The substituted `y` (red arrow) ends up bound by an inner `let` it had nothing to do with.

Now rename the inner `y` to `z`:

```
(let x y (let z 2 (+ x z)))
lazy:
---> (let z 2 (+ y z))
---> (+ y 2)
---> eval: unbound variable
```

Key Point 39 (Names of bound variables should not matter)

The two programs differ only in renaming the inner `y` to `z`, so they must mean the same thing. Yet lazy substitution gives 4 for one and an error for the other. This bug is called *variable capture*, and a substitution that can cause it is *capturing*.

Eager substitution never has this problem: it only ever substitutes numbers, and a number contains no names to capture. Lazy substitution substitutes whole expressions, which can contain free variables.

4.2.5 (SUPPLEMENT) Further Topics

Evaluation strategies. The two substitution regimes are the two classic *parameter-passing strategies*. Eager substitution is *call-by-value*, used by Racket, OCaml, Python, Java, and most languages. Lazy substitution is *call-by-name*, used for procedure parameters in Algol 60. The recomputation problem has a standard fix, *call-by-need*: substitute a reference to a shared *thunk* that evaluates `e1` the first time it is used and remembers the result. That is Haskell's evaluation strategy, and it keeps lazy's advantage (unused arguments are never evaluated) without its cost. Call-by-need gives the same results as call-by-name in a pure language; with effects, the number of times an effect happens becomes observable.

Laziness is visible. A strategy is not just an efficiency choice when evaluation can fail or loop. If `e1` loops forever, `(let x e1 10)` loops under call-by-value and returns 10 under call-by-name. This lets lazy languages define their own control structures as ordinary functions: `if` could be a function in Haskell, but not in Racket, where both branches would be evaluated before the call.

Capture in practice. Capture is not only a toy-interpreter bug. C preprocessor macros substitute text and capture freely: a macro body that declares `int tmp` silently changes the meaning of a caller's `tmp`. Racket's *hygienic* macros rename binders automatically to prevent exactly this, which is the macro-level version of the fix in the next section.

4.3 Capture-Avoiding Substitution

4.3.1 Rename, Then Substitute

The fix is *capture-avoiding substitution*: before substituting into the body of an inner `let`, rename its bound variable to a fresh name that appears nowhere else. Renaming a bound variable never changes what a program means (that was the point of the y/z example), so we are free to do it whenever it helps.

```
(let x y (let y 2 (+ x y)))
---> (let x y (let g3189 2 (+ x g3189)))    rename the inner y first
---> (let g3189 2 (+ y g3189))              the free y is no longer captured
---> (+ y 2)
---> eval: unbound variable                 same answer as the z version
```

The only question is where the fresh name comes from.

4.3.2 `gensym`: Fresh Names

`gensym` ("generate symbol") is a Racket function that returns a brand-new symbol every time it is called:

```
> (gensym)
'g3189
> (gensym)
'g3190
> (equal? (gensym) (gensym))
#f
```

A name chosen by hand, such as `'tmp`, does not work: the program might already use `tmp`, and then the renaming would create exactly the capture it was meant to avoid.

Definition 40 (Uninterned symbol)

An ordinary symbol is *interned*: `'y` is the same symbol wherever it is written, so `(equal? 'y 'y)` is `#t`. The symbol returned by `gensym` is *uninterned*: it is different from every other symbol, including one spelled the same way.

Even if a program literally contained the name `g3189`, it would not be `equal?` to the `g3189` that `gensym` returned:

```
(define g (gensym))                              ; prints as 'g3189
(equal? g (string->symbol (symbol->string g)))   ; #f
```

So a gensym can never be captured by, or capture, any name the user wrote.

Importing it into `plai-typed`. `plai-typed` does not provide `gensym`, and it cannot check a plain Racket import without knowing its type. `typed-in` imports a function from a Racket module and declares its type at the same time:

```
(require (typed-in racket/base [gensym : (-> symbol)]))
```

`(-> symbol)` means "takes no arguments and returns a symbol". The checker trusts this declaration; it does not check Racket's code.

4.3.3 `subst-lazy-v2`

Only the binding case changes. If the inner `let` binds the same name as the one being replaced, stop, as before. Otherwise:

1. let `z` be `(gensym)`, a name that appears nowhere else;
2. rename `y` to `z` in the body: `(subst-lazy (idC z) y e2)`. This cannot cause capture, because no `let` inside `e2` binds the fresh `z`;
3. substitute `v` for `x` in the renamed body.

```
(define (subst-lazy-v2 (v : ArithE) (x : symbol) (e : ArithE)) : ArithE
  (type-case ArithE e
    [numC (n) e]
    [idC (y) (if (equal? x y) v e)]
    [plusC (e1 e2) (plusC (subst-lazy-v2 v x e1) (subst-lazy-v2 v x e2))]
    [timesC (e1 e2) (timesC (subst-lazy-v2 v x e1) (subst-lazy-v2 v x e2))]
    [letC (y e1 e2)
          (if (equal? x y)
              (letC y (subst-lazy-v2 v x e1) e2)
              (let [(z (gensym))]
                (letC z
                      (subst-lazy-v2 v x e1)
                      (subst-lazy-v2 v x (subst-lazy (idC z) y e2)))))]))
```

```
(subst-lazy-v2 (idC 'y) 'x (parse '(let y 2 (+ x y))))
;; = (letC 'g3445 (numC 2) (plusC (idC 'y) (idC 'g3445)))
```

The inner `y` became `g3445`, and the `y` that came from outside is still free, so evaluation reports it as unbound. Now the `y` and `z` versions of the program agree.

Remark 41. Step 2 uses the old `subst-lazy`, which may capture in general. It is safe here because the expression being substituted, `(idC z)`, contains only the fresh name `z`, which nothing in `e2` can bind.

4.3.4 (SUPPLEMENT) Further Topics

α-equivalence. Two expressions that differ only in the names of bound variables, consistently renamed, are *α-equivalent*: `(let y 2 (+ x y))` and `(let z 2 (+ x z))` are. Semantics is defined up to α-equivalence, so an implementation may rename at will, and capture-avoiding substitution is precisely "rename to avoid a clash, then substitute". In the textbook definition [5, Section 5.3], the binder case only requires $y \neq x$ and $y \notin \mathrm{FV}(v)$; renaming is needed only when $y \in \mathrm{FV}(v)$. The lecture code renames on every binder, which is simpler and also correct.

Avoiding names altogether. Implementations often sidestep renaming by not using names for bound variables at all. With *de Bruijn indices*, a variable is replaced by the number of binders between its use and its binder: `(let y 2 (+ x y))` becomes, schematically, `(let 2 (+ x #0))`, and α-equivalent terms become identical. Substitution then shifts indices instead of renaming. Proof assistants and compilers use this or the "locally nameless" hybrid (indices for bound variables, names for free ones).

Environments. Substitution rebuilds the body for every `let` and every function call, which costs time proportional to the body's size each time. Real interpreters instead keep an *environment*, a map from names

to values, and look names up when they are used; [1, Chapter 6] makes this switch and shows that it is easy to get wrong in a way that yields dynamic scope (Section 2.4). The substitution-based interpreter remains the reference: an environment-based one is correct if it always gives the same answers.

4.4 Functions: `lambda` and Application

4.4.1 Syntax

The last part of the lecture adds functions:

syntax	meaning	abstract syntax
`(lambda x e)`	a function with parameter `x` and body `e`	`lambdaC`
`(e1 e2)`	call the function `e1` on the argument `e2`	`appC`

```
(define-type ArithE
  [numC (n : number)]
  [idC (x : symbol)]
  [plusC (e1 : ArithE) (e2 : ArithE)]
  [timesC (e1 : ArithE) (e2 : ArithE)]
  [letC (x : symbol) (e1 : ArithE) (e2 : ArithE)]
  [lambdaC (x : symbol) (e : ArithE)]
  [appC (e1 : ArithE) (e2 : ArithE)])
```

`lambdaC` *defines* a function and `appC` *uses* one, much as `letC` introduces a name and `idC` uses it. Functions take one argument; several arguments can be simulated by currying (Section 2.3).

Parsing. Any list whose head is not `+`, `*`, `let`, or `lambda` is an application:

```
(define (parse (s : s-expression)) : ArithE
  (cond
    [(s-exp-number? s) (numC (s-exp->number s))]
    [(s-exp-symbol? s) (idC (s-exp->symbol s))]
    [(s-exp-list? s)
     (let [(l (s-exp->list s))]
       (if (s-exp-symbol? (first l))
           (case (s-exp->symbol (first l))
             [(+) (plusC (parse (second l)) (parse (third l)))]
             [(*) (timesC (parse (second l)) (parse (third l)))]
             [(let) (letC (s-exp->symbol (second l)) (parse (third l)) (parse (fourth l)))]
             [(lambda) (lambdaC (s-exp->symbol (second l)) (parse (third l)))]
             [else (appC (parse (first l)) (parse (second l)))])
           ;; head is not a symbol, e.g. ((lambda x (+ x 1)) 5)
           (appC (parse (first l)) (parse (second l))))))]))
```

```
(parse '((lambda x (+ x 1)) 5))
;; = (appC (lambdaC 'x (plusC (idC 'x) (numC 1))) (numC 5))
(parse '(f 5))
;; = (appC (idC 'f) (numC 5))
```

The outer `if` matters for calls whose head is itself a list, like `((lambda x (+ x 1)) 5)`: calling `s-exp->symbol` on it would fail with `s-exp->symbol: not a symbol`.

Remark 42. With an application fallback, `'(- 1 2)` no longer fails to parse. The `else` branch reads it as a call of a function named `-`, giving `(appC (idC '-) (numC 1))` and silently dropping the `2`. Before `lambda` was added, it was `case: no matching clause`.

4.4.2 `lambda` in Substitution

A `lambda` binds its parameter in its body, exactly as `let` binds `x` in `e2`. So the `lambdaC` case of each substitution function is its `letC` case without an `e1`: stop at shadowing, and in `subst-lazy-v2` rename with `gensym` first. The application case just recurses.

```
;; in subst-eager (subst-lazy is the same with subst-lazy)
[lambdaC (y e)
         (if (equal? x y) (lambdaC y e) (lambdaC y (subst-eager v x e)))]
[appC (e1 e2) (appC (subst-eager v x e1) (subst-eager v x e2))]

;; in subst-lazy-v2
[lambdaC (y e)
         (if (equal? x y)
             (lambdaC y e)
             (let [(z (gensym))]
               (lambdaC z (subst-lazy-v2 v x (subst-lazy (idC z) y e)))))]
[appC (e1 e2) (appC (subst-lazy-v2 v x e1) (subst-lazy-v2 v x e2))]
```

Pitfall 43 (The pattern variable shadows the argument)

In `[lambdaC (y e) ...]`, the pattern variable `e` (the body) shadows `subst`'s own argument `e` (the whole `lambda`). So the shadowing branch must rebuild `(lambdaC y e)`; returning plain `e` would drop the `lambda` and return only its body. Shadowing bites the interpreter's author too.

4.4.3 A New Result Type

A program can now produce a function, for example `(lambda x (+ x 1))` or `(let f (lambda x x) f)`, and a function is not a number. So evaluation returns a Value, which is either:

```
(define-type Value
  [numV (n : number)]
  [funV (x : symbol) (e : ArithE)])
```

`+` and `*` only make sense on numbers, so they unwrap `numV` first and complain about anything else:

```
(define (num+ (v1 : Value) (v2 : Value)) : Value
  (if (and (numV? v1) (numV? v2))
      (numV (+ (numV-n v1) (numV-n v2)))
      (error 'eval "not a number")))

(define (num* (v1 : Value) (v2 : Value)) : Value
  (if (and (numV? v1) (numV? v2))
      (numV (* (numV-n v1) (numV-n v2)))
      (error 'eval "not a number")))
```

This is the first *dynamic* type error in our language: whether `(+ e1 e2)` is meaningful depends on what `e1` and `e2` evaluate to, which the interpreter only learns at run time.

4.4.4 `eval-lazy-v2`

```
(define (eval-lazy-v2 (e : ArithE)) : Value
```

```
(type-case ArithE e
  [numC (n) (numV n)]
  [idC (x) (error 'eval "unbound variable")]
  [plusC (e1 e2) (num+ (eval-lazy-v2 e1) (eval-lazy-v2 e2))]
  [timesC (e1 e2) (num* (eval-lazy-v2 e1) (eval-lazy-v2 e2))]
  [letC (y e1 e2) (eval-lazy-v2 (subst-lazy-v2 e1 y e2))]
  [lambdaC (x e) (funV x e)]
  [appC (e1 e2)
        (type-case Value (eval-lazy-v2 e1)
          [funV (x e) (eval-lazy-v2 (subst-lazy-v2 e2 x e))]
          [numV (n) (error 'eval "not a function")])]))
```

- **`lambdaC`**: a `lambda` is already a value. There is nothing to compute until it is called.
- **`appC`**: evaluate `e1`; it must be a function `(funV x e)`. Then call it the way `let` works: substitute the argument `e2` for `x` in the body `e`, and evaluate the result. `e2` is substituted unevaluated, because this is the lazy version.

```
((lambda x (+ x 1)) 5)
---> (+ 5 1)             subst 5 for x in the body
---> 6
```

Key Point 44 (`let` is application)

`((lambda x e) e2)` means the same as `(let x e2 e)`. What `lambda` adds is that the function is a *value*: it can be named with `let`, passed as an argument, and returned as a result.

program	result
`(let f (lambda x (* x 2)) (f 5))`	`(numV 10)`
`((lambda f (f 3)) (lambda x (+ x 1)))`	`(numV 4)`: a function as argument
`(((lambda x (lambda y (+ x y))) 1) 2)`	`(numV 3)`: a function as result
`((lambda x 5) y)`	`(numV 5)`: the unused argument is never evaluated
`(5 3)`	`not a function`
`(+ (lambda x x) 1)`	`not a number`

4.4.5 Capture, Again

With functions, capture-avoiding substitution matters even more. A function can return a function, so a free name can end up under another `lambda`:

```
(((lambda x (lambda y (+ x y))) y) 2)
subst-lazy:     (lambda y (+ y y)),  then 2 ---> 4          captured!
subst-lazy-v2:  (lambda g (+ y g)),  then 2 ---> (+ y 2) ---> unbound variable
```

`eval-eager` and `eval-lazy` still return a `number`, and `subst-eager` can substitute only numbers, so their `lambdaC` and `appC` cases remain errors (`"TODO: lambda"`) for now.

4.4.6 (SUPPLEMENT) Further Topics

The lambda calculus. Strip the language down to `idC`, `lambdaC`, and `appC` and you get Church's *lambda calculus*, whose only computation step is β-reduction, $(\lambda x.\, e)\ e_2 \longrightarrow e[e_2/x]$, exactly the `appC` case above. It is Turing-complete: numbers, Booleans, pairs, and recursion can all be encoded as functions. For instance, the Church numeral n is $\lambda f.\, \lambda x.\, f(f(\cdots f(x)))$ with n applications of f. Capture-avoiding substitution was invented for it; [5, Chapter 5] develops it in detail.

Why `funV` stores no environment. In `eval-lazy-v2`, a function value is just its parameter and body. That is enough only because substitution has already replaced every free variable of the body by the time the `lambda` is evaluated, as in `(((lambda x (lambda y (+ x y))) 1) 2)`, where `x` is replaced by `1` before the inner `lambda` becomes a value. An environment-based interpreter cannot do this: it must store the environment with the function, as a *closure* (Section 2.3). Forgetting to do so gives dynamic scope [1, Chapter 6].

When can capture actually happen?. Both evaluators substitute from the outside in: an outer `let` or call is processed before anything inside it. So if the whole program is closed, every expression that gets substituted is closed as well, and there is nothing to capture. Capture therefore shows up only in programs with free variables, exactly the lecture's examples, where the right answer is "unbound variable" but a capturing substitution produces a number. It becomes unavoidable in settings that substitute *open* terms: reducing inside function bodies (normal-order reduction in the lambda calculus, or a compiler inlining and simplifying a function body), and macro expansion. That is why every serious implementation of substitution is capture-avoiding.

Call-by-value application. An eager `appC` evaluates the argument first and substitutes the resulting value into the body. Under call-by-value, `((lambda x 5) y)` is an unbound-variable error instead of 5, the same eager/lazy difference as for `let` in Section 4.2. Racket, like most languages, applies functions this way.

4.5 Recitation 4: Pattern Matching in the Interpreter

In Section 3.4 we *used* Racket's `match` to write `factorial` and `fibonacci`. This recitation *implements* a small version of it: the calculator with `let` gets a `match` form, and each part of the interpreter is extended in turn: the types, `parse`, `subst`, and `eval`. The file starts with the `gensym` import of Section 4.3, since its `let` uses capture-avoiding lazy substitution.

4.5.1 The New Form

A pattern-matching expression looks like

```
(match e (n1 e1) (n2 e2) ... (_ en))
```

where each `ni` is a number literal and _ is the wildcard. To evaluate it:

1. evaluate `e` to a number, once;
2. try the clauses top to bottom: `ni` matches if it equals that number, and _ matches anything;
3. the *first* matching clause wins, and only its expression is evaluated;
4. if no clause matches, it is an error.

This is Racket's `match` restricted to literals and _, just enough for `factorial`- and `fibonacci`-style definitions. There are no pattern variables, so a clause never binds a name; that matters for substitution.

4.5.2 Types

A clause pairs a pattern with an expression. The two kinds of clause carry different data (a number pattern has a number, the wildcard nothing), so each gets its own variant:

```
(define-type PatternE
  [numP (n : number) (e : ArithE)]
  [anyP (e : ArithE)])

(define-type ArithE
  [numC (n : number)]
  [plusC (e1 : ArithE) (e2 : ArithE)]
  [timesC (e1 : ArithE) (e2 : ArithE)]
  [letC (x : symbol) (e1 : ArithE) (e2 : ArithE)]
  [idC (x : symbol)]
  [patternC (e : ArithE) (patterns : (listof PatternE))])
```

`patternC` holds the expression being matched, the *scrutinee*, and the clauses in order. The two types are *mutually recursive*: an ArithE can contain PatternEs, and each PatternE contains an ArithE. `define-type` allows that, in either order.

Examples. The handout's four examples, in ML-like notation:

	program	expected
1	`match 5 with | 5 -> 0 | 6 -> 1`	0
2	`match (1 + 2) with | 3 -> 1 | _ -> x`	1
3	`match x with | 0 -> z | _ -> (match y with | 0 -> x | 1 -> y)`	unbound variable
4	`let x = 1 in let y = 1 in` (the `match` of 3, last clause `y + 1`)	2

```
(define example-2 : ArithE
  (patternC (plusC (numC 1) (numC 2))
            (list (numP 3 (numC 1))
                  (anyP (idC 'x)))))

(define example-4 : ArithE
  (letC 'x (numC 1)
        (letC 'y (numC 1)
              (patternC (idC 'x)
                        (list (numP 0 (idC 'z))
                              (anyP (patternC (idC 'y)
                                              (list (numP 0 (idC 'x))
                                                    (numP 1 (plusC (numC 1) (idC 'y))))))))))))
```

Examples 2 and 4 both contain a name that nothing binds (`x` and `z`), yet neither is an error, because that clause is never chosen. In example 3 the error comes from the scrutinee `x`, which is always evaluated.

4.5.3 Parsing

The concrete syntax is `(match e clause ...)`, so after the `match` keyword, `(second l)` is the scrutinee and `(rest (rest l))` is the list of clauses. A helper turns that list of s-expressions into a list of PatternEs. Each clause is a two-element list `(pattern expr)`, and the pattern decides which variant to build:

```
(define (parse-patterns (s-pats : (listof s-expression))) : (listof PatternE)
  (cond
    [(empty? s-pats) empty]
    [else
     (let [(s-pat (s-exp->list (first s-pats)))]
       (cond
         [(s-exp-number? (first s-pat))
          (cons (numP (s-exp->number (first s-pat)) (parse (second s-pat)))
                (parse-patterns (rest s-pats)))]
         [(equal? (s-exp->symbol (first s-pat)) '_)
          (list (anyP (parse (second s-pat))))]))]))

(define (parse (s : s-expression)) : ArithE
  (cond
    [(s-exp-number? s) (numC (s-exp->number s))]
    [(s-exp-symbol? s) (idC (s-exp->symbol s))]
    [(s-exp-list? s)
     (let [(l (s-exp->list s))]
       (case (s-exp->symbol (first l))
         [(+) (plusC (parse (second l)) (parse (third l)))]
         [(*) (timesC (parse (second l)) (parse (third l)))]
         [(let) (letC (s-exp->symbol (second l)) (parse (third l)) (parse (fourth l)))]
         [(match) (patternC (parse (second l)) (parse-patterns (rest (rest l))))]))]))
```

Three details of the TA solution:

- A number clause is consed onto the rest, but a `_` clause returns `(list (anyP ...))`: parsing *stops* at the first wildcard. Any clause after it could never be reached, so dropping it does not change what the program means: `(parse '(match 1 (_ 0) (2 3)))` has one clause.

- The pattern is *not* parsed as an expression; it must be a number literal or _. Anything else is an error, though not one this function raises itself: `(match 3 ((+ 1 2) 0))` fails in `s-exp->symbol` (`not a symbol`), and `(match 3 (n 0))` falls off the end of the inner `cond` (`no matching clause`).
- `parse-patterns` and `parse` call each other, just as the two types refer to each other. `match` is a symbol in *our* language, matched by `case` like `+` and `let`.

4.5.4 Substitution

Exercise 45 (Why substitute into `match`?)

Why do we need to define substitution for pattern-matching expressions?

Solution. Because `let` is implemented by substitution, and the body of a `let` can contain a `match`. In example 4, the `x` and `y` bound by the `let`s are used in the scrutinees and in the clause bodies. If `subst` stopped at a `patternC`, those `idC`s would never be replaced and `eval` would report them as unbound. (In `plai-typed` the question does not really arise: `type-case` rejects a `subst` that has no `patternC` case.)

Substituting into a clause means substituting into its expression; the pattern itself is a number or _, with no names in it:

```
(define (subst-pattern (erep : ArithE) (x : symbol)) : (PatternE -> PatternE)
  (lambda (p)
    (type-case PatternE p
      [numP (n e) (numP n (subst erep x e))]
      [anyP (e) (anyP (subst erep x e))])))
```

`subst-pattern` is *curried*: it takes `erep` and `x` and returns a function from one clause to one clause. That is exactly the shape `map` wants, so the `patternC` case is one line:

```
(define (subst (erep : ArithE) (x : symbol) (e : ArithE)) : ArithE
  (type-case ArithE e
    [idC (y) (if (equal? x y) erep (idC y))]
    [numC (n) (numC n)]
    [plusC (e1 e2) (plusC (subst erep x e1) (subst erep x e2))]
    [timesC (e1 e2) (timesC (subst erep x e1) (subst erep x e2))]
    [patternC (e pats) (patternC (subst erep x e) (map (subst-pattern erep x) pats))]
    [letC (y e1 e2)
          (cond
            [(equal? x y) (letC y (subst erep x e1) e2)]
            [else
             (let [(z (gensym))]
               (letC z (subst erep x e1) (subst erep x (subst (idC z) y e2))))])]))
```

The `letC` case is lecture 4's capture-avoiding `subst-lazy-v2`. Note what the `patternC` case does *not* need: no shadowing check and no renaming. Those exist for `letC` because a `let` binds a name; our patterns bind nothing, so a clause body sees exactly the same names as the `match` around it, like the children of `plusC`.

4.5.5 Evaluation

The helper `get-pat-expr` takes the number the scrutinee evaluated to and the clauses, and returns the *expression* of the first clause that matches:

```
(define (get-pat-expr (num : number) (pats : (listof PatternE))) : ArithE
  (cond
    [(empty? pats) (error 'get-pat-expr "unmatched expression")]
    [else (type-case PatternE (first pats)
            [numP (n e) (cond [(equal? num n) e] [else (get-pat-expr num (rest pats))])]
            [anyP (e) e])]))

(define (eval (e : ArithE)) : number
  (type-case ArithE e
    [numC (n) n]
    [plusC (e1 e2) (+ (eval e1) (eval e2))]
    [timesC (e1 e2) (* (eval e1) (eval e2))]
    [patternC (e pats) (eval (get-pat-expr (eval e) pats))]
    [letC (x e1 e2) (eval (subst e1 x e2))]
    [idC (x) (error 'eval "unbound variable!")]))
```

`eval` evaluates the scrutinee once, picks the clause, and evaluates only that clause's expression.

Why return the expression, not its value?. `get-pat-expr` could evaluate every clause and return numbers. That would be wrong, not just slow: example 4 has a clause `(0 z)`, and evaluating it would raise "unbound variable" even though the scrutinee is 1 and the clause is never chosen. Returning the unevaluated ArithE means only the chosen clause is evaluated, just as only one branch of an `if` is.

Tracing example 4. `g1` stands for whatever name `gensym` returns.

```
(let x 1 (let y 1 (match x (0 z) (_ (match y (0 x) (1 (+ 1 y)))))))
letC x:  subst (numC 1) for x in the body.
         The inner let binds y, not x, so y is renamed to g1 first.
---> (let g1 1 (match 1 (0 z) (_ (match g1 (0 1) (1 (+ 1 g1))))))
letC g1: subst (numC 1) for g1 in the body
---> (match 1 (0 z) (_ (match 1 (0 1) (1 (+ 1 1)))))
patternC: scrutinee = 1; (0 z) fails; _ matches
---> (match 1 (0 1) (1 (+ 1 1)))
patternC: scrutinee = 1; (0 1) fails; (1 ...) matches
---> (+ 1 1)
---> 2
```

The clause `(0 z)` is skipped without being evaluated, which is why the free `z` is harmless. Example 3 fails at its first step: the scrutinee is `(idC 'x)`, no `let` has replaced it, and `eval` reports it as unbound.

Clause order. Because `get-pat-expr` walks the list from the front and stops at the first match, order matters, and a clause after _ is dead code:

```
(test (eval (parse '(match 5 (_ 0) (5 1)))) 0)
(test (eval (parse '(match 5 (5 1) (_ 0)))) 1)
(test (eval (parse '(match 5 (5 1) (5 2)))) 1)
(test (eval (parse '(match (* 2 3) (5 0) (6 1) (_ 2)))) 1)
(test/exn (eval (parse '(match 7 (5 0) (6 1)))) "unmatched expression")
```

The last test is the run-time "no matching clause" error of Section 3.4, now in our own language. Nothing can check for it ahead of time: whether a clause matches depends on a number known only when the program runs.

4.5.6 (SUPPLEMENT) Further Topics

Variable patterns. Racket's `match` also allows a name as a pattern: it matches anything, like `_`, and binds the name to the matched value. With that, `factorial`'s clause could read `(n (* n ...))`. Add it as a third kind of clause, and a clause `(n e)` behaves like `(let n v e)`, where `v` is the value of the scrutinee:

```
(define-type PatternE
  [numP (n : number) (e : ArithE)]
  [anyP (e : ArithE)]
  [varP (x : symbol) (e : ArithE)])

;; new clause in get-pat-expr: substitute the scrutinee's value
[varP (y e) (subst (numC num) y e)]

;; new clause in subst-pattern: a varP clause is a binder, like letC
[varP (y e)
      (if (equal? x y)
          (varP y e)
          (let [(z (gensym))]
            (varP z (subst erep x (subst (idC z) y e)))))]
```

Only the clause body is under the binder; the scrutinee is outside it, so the `patternC` case of `subst` still substitutes into the scrutinee unconditionally, as `letC` always substitutes into `e1`. Each piece is needed:

program	correct	without the piece
`(let y 10 (match (+ 1 2) (0 0) (n (+ n y))))`	13	(baseline)
`(let n 5 (match 7 (n (* n 2))))`	14	no shadowing check: 10
`(let x n (match 3 (n x)))`	unbound	no renaming: 3

In the second program the clause's `n` shadows the outer one, so the `let` must not substitute 5 into the clause body. In the third, `let` substitutes lazily, so the free `n` would move under the clause that binds `n` and be captured: the `(let x y (let y 2 (+ x y)))` example of Section 4.2, with a pattern in place of the inner `let`. In general, *every* construct that binds a name needs the same two treatments in substitution.

Compiling pattern matching. Real implementations do not try clauses one by one at run time. A compiler turns a `match` into a *decision tree* or a *backtracking automaton* that tests each part of the scrutinee at most once; for integer literals this becomes a jump table or a binary search. The same analysis reports unreachable clauses, like those after `_` that `parse-patterns` silently drops, and, with types, non-exhaustive matches (Section 3.4).

Bibliography

[1] S. Krishnamurthi. *Programming Languages: Application and Interpretation*, 2nd edition. 2017. https://cs.brown.edu/~sk/Publications/Books/ProgLangs/

[2] M. Felleisen, R. B. Findler, M. Flatt, and S. Krishnamurthi. *How to Design Programs*, 2nd edition. MIT Press, 2018. https://htdp.org

[3] H. Abelson and G. J. Sussman. *Structure and Interpretation of Computer Programs*, 2nd edition. MIT Press, 1996.

[4] M. Flatt, R. B. Findler, and PLT. *The Racket Guide*. https://docs.racket-lang.org/guide/

[5] B. C. Pierce. *Types and Programming Languages*. MIT Press, 2002.

[6] J. C. Reynolds. Definitional interpreters for higher-order programming languages. In *Proceedings of the ACM Annual Conference*, 1972.